CSV Data Cleaning

จัดทำโดย

6604062630510 วรรษชล ลาวัง

เสนอ

อาจารย์อนุสรณ์ วงษ์สนิท

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา Unix Tools

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

รายละเอียด

1. ชื่อไฟล์สคริปต์: clean_csv.sh

ประเภท: Bash script

วัตถุประสงค์: ทำความสะอาดไฟล์ CSV โดยตรวจสอบความถูกต้องของข้อมูล, จัดการคอลัมน์, จัดการค่า missing, outlier และทำ normalization ให้ข้อมูลพร้อมสำหรับการวิเคราะห์ต่อไป

ฟังก์ชันหลัก:

- ลบหรือกรองข้อมูลที่ไม่จำเป็น เช่น ค่าซ้ำ, missing values
- แยก column/field ที่ต้องการจาก dataset
- คำนวณสถิติพื้นฐาน เช่น ค่าเฉลี่ย, ค่าสูงสุด, ค่าต่ำสุด
- จัดการ missing values ตามที่ผู้ใช้เลือก เช่น drop, fill ด้วย mean, median modeหรือค่าคงที่
- ตรวจสอบความถูกต้องของข้อมูล เช่น type check, range check
- จัดการ outliers ใน numeric columns
- Normalize / scale ข้อมูลตัวเลข
- บันทึก dataset ที่ผ่านการทำความสะอาดแล้วเป็นไฟล์ใหม่

2. เครื่องมือที่ใช้

ระบบปฏิบัติการ: Unix / Linux
เครื่องมือและคำสั่งหลัก:

ประเภท	เครื่องมือ / คำสั่ง	หน้าที่

Filter	grep, cut, awk, sed	ดึงหรือกรองข้อมูล, แยก column, แก้ไขค่าในไฟล์
Statistical	awk	คำนวณค่าเฉลี่ย, min, max, sum, median
File Handling	cat, sort, uniq, wc, mv	ตรวจสอบค่าซ้ำ, นับจำนวน row/column
Text Editing	vi, sed	แก้ไขไฟล์ CSV
Interactive Menu	echo, read, select	ให้ผู้ใช้เลือก option สำหรับ data cleaning

Dataset: xxx_dataset.csv โดยมีตัวอย่างดังนี้ (ไฟล์เต็มสามารถดูได้ที่ส่วนอ้างอิง)

```
gender,age,hypertension,heart_disease,Marriage,work_type,Living_type,avg_glucose,bmi,smoking_status,illness
Male,67,0,1,Yes,Private,Urban,228.69,36.6,formerly smoked,1
Male,1000,0,1,Yes,Private,Rural,105.92,32.5,never smoked,1
Female,49,0,0,Yes,Private,Urban,171.5000,34.4,smokes,1
Female,79,1,0,Yes,Self-employed,Rural,174.12,24,never smoked,1
Male,81,0,0,Yes,Private,Urban,186.21,29,formerly smoked,1
Male,74,1,1,Yes,Private,Rural,70.09,27.4,never smoked,1
Female,69p,0,0,No,Private,Urban,94.39,22.8,never smoked,1
Female,78,0,0,Yes,Private,Urban,58.57,24.2,Unknown,1
Female,ff,1,0,Yes,Private,Rural,80.43,29.7,never smoked,1
Female,61,0,1,Yes,Govt job,Rural,120.46,36.8,smokes,1
```

3. ฟังก์ชันการทำงานในสคริปต์

สคริปต์มี menu ให้ผู้ใช้เลือกวิธีการทำความสะอาดข้อมูล โดยแต่ละ option มีรายละเอียดดังนี้

1. **Show sample data:**

   แสดงข้อมูลตัวอย่าง 10 แถวแรก เพื่อให้ผู้ใช้เห็นโครงสร้าง dataset

2. **View full data:**

   ใช้ less เพื่อดูข้อมูลทั้งหมด สามารถออกด้วย q

3. **View / modify column types:**

   ตรวจสอบชนิดข้อมูลของ column (Numeric / Categorical) เพื่อใช้ในการทำความสะอาดแต่ละประเภทที่ไม่เหมือนกัน และสามารถแก้ไขได้

4. **Check numeric columns:**

   ตรวจสอบไม่ให้ค่าของ column ประเภทตัวเลขไม่มีตัวอักษรหรืออักขระอื่น ๆ ผสม

5. **Handle missing values:**

   จัดการค่าที่หายไป โดยเลือกวิธี drop หรือเติมด้วยค่า mean/median/mode/constant

6. **Handle outliers:**

   ตรวจจับและจัดการค่า outlier ใน numeric columns โดยมี 2 method คือ IQR method กับ Z-score method

7. **Normalize / scale numeric features:**

   ทำ normalization หรือ scaling โดยมี 2 methof คือ Min-Max scaling กับ Z-score

8. **Check data validity / accuracy:**

   ตรวจสอบความถูกต้องของข้อมูล เช่น ตรวจสอบ range, type check, consistency

9. **Remove duplicate rows:**

   ลบแถวที่ซ้ำกันออกจาก dataset

10. **Manual edit data:**

    เปิดไฟล์ CSV ให้ผู้ใช้ด้วยตัวเอง โดยสามารถเลือกได้ว่าจะแก้ไขค่า ๆ นึงทั้ง column หรรือจะเจาะจงเฉพาะ row

11. **Drop column:**

    ลบ column ที่ไม่ต้องการ

12. **Exit:**

    ออกจากสคริปต์

4. การใช้งานสคริปต์

1. วางไฟล์ clean_csv.sh ในโฟลเดอร์ที่เก็บ CSV
2. ให้สิทธิ์รันสคริปต์:
    - chmod +x clean_csv.sh
3. เลือกไฟล์ CSV และรันสคริปต์
    - ./clean_csv.sh file.csv
4. เลือกเมนูที่ต้องการทำความสะอาดข้อมูล
5. ได้ไฟล์ใหม่หลังจากทำความสะอาดเสร็จ จะได้ไฟล์ใหม่โดยจะมีชื่อต่อท้ายว่า _clean.csv เช่น ถ้าไฟล์เดิมชื่อว่า xxx_dataset.csv จะได้ไฟล์ใหม่ว่า xxx_dataset_clean.csv

อธิบายการทำงานของสคริปต์

1. ส่วน validation

- **ตรวจสอบจำนวนอาร์กิวเมนต์ที่ส่งเข้ามาว่าเท่ากับ 1 หรือไม่**

```
if [ $# -ne 1 ]; then
    echo "Error : $0 <file.csv>"
    exit 1
fi
```

- **กำหนดชื่อไฟล์อินพุตและไฟล์เอาต์พุต**

```
INPUT_FILE="$1"
OUT_FILE="${INPUT_FILE%.csv}_clean.csv"
```

  - $1 คืออาร์กิวเมนต์ตัวแรก (ชื่อไฟล์ CSV)
  - OUT_FILE เป็นชื่อไฟล์ใหม่ที่เอา _clean ต่อท้ายก่อนนามสกุล .csv

- **ตรวจสอบว่าไฟล์สามารถอ่านได้หรือไม่**

```
if [ ! -r "$INPUT_FILE" ]; then
    echo "Error: File '$INPUT_FILE' is not readable"
    exit 1
fi
```

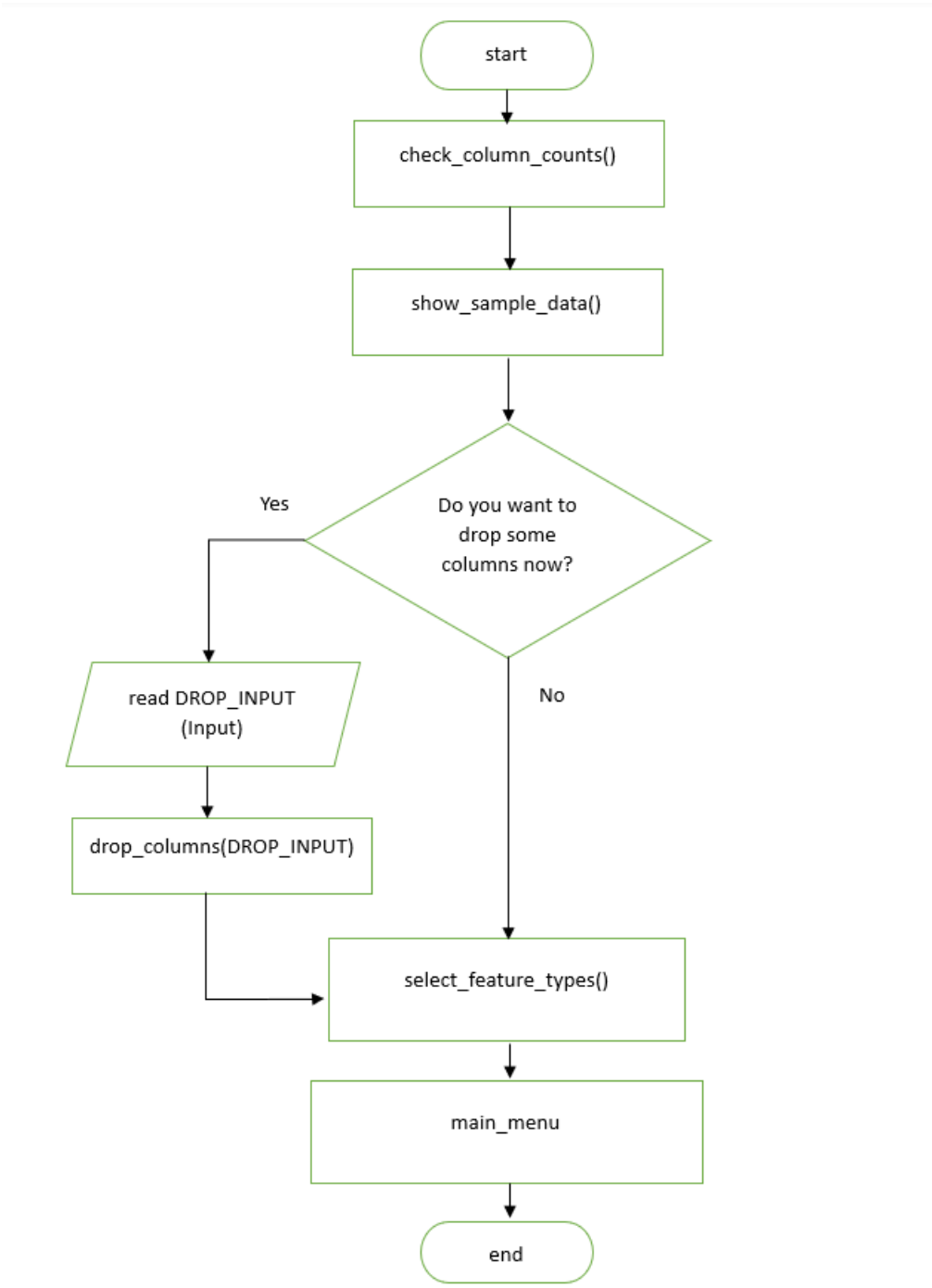
- **ตรวจสอบว่าไฟล์สามารถอ่านได้หรือไม่**

```
if [ ! -s "$INPUT_FILE" ]; then
    echo "Error: File '$INPUT_FILE' is empty!"
    exit 1
fi
```

- **ตรวจสอบว่าไฟล์ว่างหรือไม่**

```
if [ "${INPUT_FILE##*.}" != "csv" ] && [ "${INPUT_FILE##*.}" != "CSV" ]; then
    echo "Error: File must have .csv extension!"
    exit 1
fi
```

2. การทำงานของส่วนหลัก (ไม่รวมฟังก์ชัน)



```
#!/bin/bash

#--functions--
        :
        :
        :
        :
#----------
check_column_counts
show_sample_data
echo ""
if confirm_input "Do you want to drop some columns?"; then
    read -p "Enter column numbers to drop (comma separated) or 0 to keep all: " DROP_INPUT
    drop_columns "$DROP_INPUT"
fi


select_feature_types
main_menu
~
```

ลำดับการทำงานของส่วนหลัก (ไม่รวมฟังก์ชัน)

1. check_column_counts

- เป็นฟังก์ชันเริ่มต้นของสคริปต์ ทำหน้าที่ ตรวจสอบจำนวนคอลัมน์ในแต่ละแถวของไฟล์ CSV
- เพื่อดูว่ามีแถวใดที่มีจำนวนคอลัมน์ไม่เท่ากับ header หรือไม่
- ช่วยป้องกัน error ระหว่างการ clean ข้อมูลในขั้นตอนต่อไป

2. show_sample_data

- ใช้สำหรับ แสดงตัวอย่างข้อมูลบางส่วนของ dataset (เช่น 5–10 แถวแรก)
- เพื่อให้ผู้ใช้เห็นภาพรวมของไฟล์ที่กำลังจะทำการ clean เช่น ชื่อคอลัมน์ และรูปแบบข้อมูลในแต่ละ field
- ข้อมูลนี้ช่วยให้ผู้ใช้ตัดสินใจได้ว่าจะลบหรือแก้ไขคอลัมน์ใด
- confirm_input "Do you want to drop some columns?" เป็นส่วนของการถามผู้ใช้ว่า ต้องการลบคอลัมน์บางส่วนตอนนี้หรือไม่

ฟังก์ชัน confirm_input จะให้ผู้ใช้ตอบเพียง y หรือ n

- ถ้าผู้ใช้ตอบ y จะเข้าสู่ขั้นตอนการเลือกคอลัมน์เพื่อลบ
- ถ้าผู้ใช้ตอบ n ข้ามไปยังขั้นตอนถัดไปทันที

3. drop_columns "$DROP_INPUT"

- ถ้าผู้ใช้ตอบ “yes” ระบบจะให้พิมพ์หมายเลขคอลัมน์ที่ต้องการลบ (เช่น 2,4,6)
- ฟังก์ชัน drop_columns จะทำการ ลบคอลัมน์ที่ระบุออกจาก dataset
- ถ้าพิมพ์ 0 หมายถึงไม่ต้องลบคอลัมน์ใดเลย

4. select_feature_types

- หลังจากตรวจสอบและจัดการคอลัมน์แล้ว ระบบจะเข้าสู่ขั้นตอนการ ระบุชนิดข้อมูลของแต่ละคอลัมน์
- เช่น คอลัมน์ที่เป็นตัวเลข (Numeric) หรือข้อมูลเชิงหมวดหมู่ (Categorical)
- การแยกประเภทนี้สำคัญต่อขั้นตอนการ handle missing, outlier และ normalization

5. main_menu

- เป็นฟังก์ชันหลักของโปรแกรม ทำหน้าที่แสดง เมนูการทำความสะอาดข้อมูลทั้งหมด
- เช่น การดูข้อมูลทั้งหมด, จัดการ missing values, ลบ outliers, normalize, ลบค่าซ้ำ หรือแก้ไขข้อมูลด้วยตนเอง
- ผู้ใช้สามารถเลือกเมนูจากตัวเลขที่แสดง เพื่อดำเนินการแต่ละขั้นตอนตามลำดับ

ตัวอย่างการแสดงผล

```
=== Checking column counts (expected 11) ===
Some rows do not have the expected number of columns.
This may indicate missing or extra values in a row.

--------------------------------------------------
Found rows with incorrect number of columns (first column is row number):

gender age hypertension heart_disease Marriage work_type Living_type avg_glucose bmi smoking_status illness
->
393 Male 0 0 No Private Rural 123.79 20.5 Unknown 0
403 Male 44 0 0 Yes Private Urban 99.34 33.1 never smoked

Choose how to handle rows with incorrect column count:
1) Remove these rows
2) Exit script to fix manually
Enter your choice (1 or 2): 1

Rows with incorrect column count removed and saved to xxx_dataset_clean.csv.
Press any key to continue...
```

```
=== Sample data from file xxx_dataset_clean.csv (401 rows, include column name)===
--------------------------------------------------
gender  age   hypertension  heart_disease  Marriage  work_type      Living_type  avg_glucose  bmi   smoking_status   illness
Male    67    0             1              Yes       Private        Urban        228.69       36.6  formerly smoked  1
Male    1000  0             1              Yes       Private        Rural        105.92       32.5  never smoked     1
Female  49    0             0              Yes       Private        Urban        171.5000     34.4  smokes           1
Female  79    1             0              Yes       Self-employed  Rural        174.12       24    never smoked     1
Male    81    0             0              Yes       Private        Urban        186.21       29    formerly smoked  1
Male    74    1             1              Yes       Private        Rural        70.09        27.4  never smoked     1
Female  69p   0             0              No        Private        Urban        94.39        22.8  never smoked     1
Female  78    0             0              Yes       Private        Urban        58.57        24.2  Unknown          1
Female  ff    1             0              Yes       Private        Rural        80.43        29.7  never smoked     1
--------------------------------------------------

Press any key to continue...
Do you want to drop some columns? (y/n):
```

```
Do you want to drop some columns? (y/n): y
Enter column numbers to drop (comma separated) or 0 to keep all: 5,6
You are about to drop columns: 5,6
Are you sure? (y/n): y
-- Columns dropped successfully. Updated sample --
gender  age   hypertension  heart_disease  Living_type  avg_glucose  bmi   smoking_status   illness
Male    67    0             1              Urban        228.69       36.6  formerly smoked  1
Male    1000  0             1              Rural        105.92       32.5  never smoked     1
Female  49    0             0              Urban        171.5000     34.4  smokes           1
Female  79    1             0              Rural        174.12       24    never smoked     1

Press any key to continue...

=== Specify column types for each feature ===
Enter 1 for Numeric  |   2 for Categorical

Column 1 - gender : 2
Column 2 - age : 1
Column 3 - hypertension : 2
Column 4 - heart_disease : 2
Column 5 - Living_type : 2
Column 6 - avg_glucose : 1
Column 7 - bmi : 1
Column 8 - smoking_status : 2
Column 9 - illness : 3
! Invalid input! Please enter 1 or 2 !
Column 9 - illness : 2

-- Column types saved --
-> Numeric columns: age,avg_glucose,bmi
-> Categorical columns: gender,hypertension,heart_disease,Living_type,smoking_status,illness

Do you want to modify column types? (y/n): n
-- Keeping previous column type selection --
Press any key to continue...
```

```
==============================
      Data Cleaning Menu
==============================
File: xxx_dataset_clean.csv (401 rows, include column name)

1) Show sample data
2) View full data (press q to exit)
3) View / modify column types (Numeric / Categorical)
4) Check numeric columns
5) Handle missing values (Numeric / Categorical)
6) Handle outliers (Numeric only)
7) Normalize / scale numeric features
8) Check data validity / accuracy
9) Remove duplicate rows
10) Manual edit data
11) Drop column
12) Exit
==============================
Choose an option:
```

- **ฟังก์ชั่น check_column_counts**

```
check_column_counts() {
    local file="$INPUT_FILE"
    local delim=","

    if head -1 "$file" | grep -q ";"; then
        delim=";"
    elif head -1 "$file" | grep -q "|"; then
        delim="|"
    elif head -1 "$file" | grep -q $'\t'; then
```

```
        delim=$'\t'
fi

DELIM="$delim"
TT_COLS=$(head -1 "$file" | awk -F"$DELIM" '{print NF}')



MIS_LINES=$(awk -F"$DELIM" -v num="$TT_COLS" 'NF!=num {print NR, $0}' "$file")

if [ -z "$MIS_LINES" ]; then
        return
fi

echo ""
echo "=== Checking column counts (expected $TT_COLS) ==="
echo "Some rows do not have the expected number of columns."
echo "This may indicate missing or extra values in a row."
echo ""
echo "-----------------------------------------------"
echo "Found rows with incorrect number of columns (first column is row number):"
echo ""
```

```
head -1 "$file" | tr "$DELIM" " "
echo "->"
echo "$MIS_LINES" | tr "$DELIM" " "
echo ""


while true; do
    echo "Choose how to handle rows with incorrect column count:"
    echo "1) Remove these rows"
    echo "2) Exit script to fix manually"
    read -p "Enter your choice (1 or 2): " ch


    if [ "$ch" = "1" ]; then
        awk -F"$DELIM" -v num="$TT_COLS" 'NF==num' "$file" > "${OUT_FILE}.tmp"
        mv "${OUT_FILE}.tmp" "$OUT_FILE"
        echo ""
        echo "Rows with incorrect column count removed and saved to $OUT_FILE."
        break
    elif [ "$ch" = "2" ]; then
        echo "Exiting script. Please fix the file and try again."
        exit 1
    else
```

```
                echo "Invalid choice! Enter 1 or 2."
                echo ""
            fi
    done

    read -n1 -r -p "Press any key to continue..."
}
```

ฟังก์ชัน check_column_counts() ใช้ตรวจสอบว่าไฟล์ข้อมูล CSV ที่นำเข้า ($INPUT_FILE) มีจำนวนคอลัมน์เท่ากันทุกแถวหรือไม่ เพื่อป้องกันปัญหาจากข้อมูลหายหรือเกินในบางบรรทัด ซึ่งอาจทำให้การประมวลผลภายหลังผิดพลาด

หลักการทำงานโดยรวม

1. ตรวจจับตัวคั่นคอลัมน์ (Delimiter Detection)
    - ใช้คำสั่ง head -1 "$file" | grep -q ";"
    - เพื่อดูว่าไฟล์ใช้ ,, ;, | หรือ tab เป็นตัวคั่น โดยตรวจจากบรรทัดแรก (head -1) จากนั้นเก็บค่าตัวคั่นไว้ในตัวแปร DELIM.
2. คำนวณจำนวนคอลัมน์ที่คาดหวัง
    - ใช้คำสั่ง awk -F"$DELIM" '{print NF}' เพื่อนับจำนวนฟิลด์ (Field count) ในบรรทัดแรก และเก็บในตัวแปร TT_COLS.
3. ตรวจหาบรรทัดที่มีจำนวนคอลัมน์ไม่เท่ากัน
    - ใช้ awk ตรวจทุกแถวในไฟล์: awk -F"$DELIM" -v num="$TT_COLS" 'NF!=num {print NR, $0}'
    - NF คือจำนวนฟิลด์ในแต่ละบรรทัด, NR คือหมายเลขบรรทัด

- ผลลัพธ์ที่ได้คือบรรทัดที่คอลัมน์ไม่ตรงจำนวนที่คาดไว้

4. แสดงผลบรรทัดที่มีปัญหา
    - ใช้ echo และ tr "$DELIM" " " เพื่อเปลี่ยนตัวคั่นเป็นช่องว่างให้อ่านง่ายขึ้น
5. ให้ผู้ใช้เลือกวิธีจัดการ
    - ตัวเลือก 1: ลบบรรทัดที่ผิดพลาด
        - awk -F"$DELIM" -v num="$TT_COLS" 'NF==num' "$file" > "${OUT_FILE}.tmp"
        - mv "${OUT_FILE}.tmp" "$OUT_FILE"
        - ใช้ awk เก็บเฉพาะบรรทัดที่มีจำนวนคอลัมน์ถูกต้อง แล้วใช้ mv ย้ายไฟล์ชั่วคราวไปแทนที่ไฟล์ผลลัพธ์
    - ตัวเลือก 2: ออกจากสคริปต์เพื่อให้ผู้ใช้แก้ไขเอง
6. หยุดรอผู้ใช้ก่อนดำเนินการต่อ
    - ใช้คำสั่ง read -n1 -r -p "Press any key to continue..." เพื่อหยุดรอจนผู้ใช้กดปุ่ม

- **ฟังก์ชั่น show_sample_data**

```
show_sample_data() {
    local file="$OUT_FILE"
    echo ""
    echo ""
    echo "=== Sample data from file $file ($(cat $file | wc -l) rows, include column name)==="
    echo "--------------------------------------------------"
    head -n  10 "$file" | column -ts "$DELIM"
    echo "--------------------------------------------------"
    echo ""

    HEADER=$(head -1 "$file")
    IFS="$DELIM" read -ra COLS <<< "$HEADER"

    read -n1 -r -p "Press any key to continue..."
}
```

ฟังก์ชัน show_sample_data() ใช้สำหรับแสดงตัวอย่างข้อมูลจากไฟล์ CSV หลังการประมวลผล เพื่อให้ผู้ใช้ตรวจสอบความถูกต้องของข้อมูลและคอลัมน์เบื้องต้นก่อนดำเนินการขั้นต่อไป

หลักการทำงานโดยรวม

1. กำหนดไฟล์ที่จะใช้แสดงข้อมูล
    - ใช้ตัวแปร local file="$OUT_FILE" เพื่อระบุไฟล์ CSV ที่ผ่านการประมวลผลก่อนหน้านี้ให้เป็นเป้าหมายของการแสดงผล

2. แสดงจำนวนแถวทั้งหมดในไฟล์
    - ใช้คำสั่ง cat $file | wc -l เพื่ออ่านเนื้อหาทั้งหมดของไฟล์ (cat) และนับจำนวนบรรทัด (wc -l) ซึ่งแสดงจำนวนแถวรวมในไฟล์
3. แสดงตัวอย่างข้อมูล 10 แถวแรก
    - ใช้ head -n 10 "$file" | column -ts "$DELIM"
    - head -n 10 แสดงเฉพาะ 10 แถวแรกของไฟล์
    - column -ts "$DELIM" จัดรูปแบบให้อยู่ในตาราง โดยใช้ตัวคั่นคอลัมน์ $DELIM (เช่น , หรือ ;) เพื่อให้อ่านง่ายขึ้น
4. อ่านชื่อคอลัมน์จากบรรทัดแรกของไฟล์
    - HEADER=$(head -1 "$file")
    - IFS="$DELIM" read -ra COLS <<< "$HEADER"
    - head -1 ดึงบรรทัดแรกซึ่งเป็นชื่อคอลัมน์
    - IFS (Internal Field Separator) ใช้แยกชื่อคอลัมน์ตามตัวคั่น แล้วเก็บไว้ในอาเรย์ COLS เพื่อใช้งานภายหลัง
5. หยุดรอผู้ใช้ก่อนดำเนินการต่อ
    - ใช้คำสั่ง read -n1 -r -p "Press any key to continue..." เพื่อรอให้ผู้ใช้ตรวจสอบข้อมูลตัวอย่างก่อนดำเนินการขั้นต่อไป

- **ฟังก์ชัน drop_columns**

```
drop_columns() {
    local file="$OUT_FILE"
    local input="$1"
    local delim="$DELIM"

    if [ "$input" = "0" ]; then
        echo "Keeping all columns (no drop)."
        cp "$file" "$OUT_FILE"
        return
    fi

    if ! echo "$input" | grep -Eq '^[0-9]+(,[0-9]+)*$'; then
        echo "! Please enter column numbers separated by commas (e.g. 1,3,5) !"
        return
    fi

    local total_cols=${#COLS[@]}
    for num in $(echo "$input" | tr ',' ' '); do
        if [ "$num" -lt 1 ] || [ "$num" -gt "$total_cols" ]; then
            echo "! Column number $num is out of range (1–$total_cols). !"
```

```
            return
        fi
done


echo "You are about to drop columns: $input"
read -p "Are you sure? (y/n): " cf
if [ "$cf" = "y" ] || [ "$cf" = "Y" ]; then


        IFS=',' read -ra DROP_ARR <<< "$input"
        KEEP_ARR=()



        for id in "${!COLS[@]}"; do
            col_id=$((id+1))
            skip=false
            for drop in "${DROP_ARR[@]}"; do
                drop=$(echo "$drop" | xargs)
                if [ "$col_id" -eq "$drop" ]; then
                    skip=true
                    break
                fi
```

```
        done
        if [ "$skip" = false ]; then
            KEEP_ARR+=("$col_id")
        fi
    done

    CUT_COLS=$(IFS=, ; echo "${KEEP_ARR[*]}")

    cut -d"$delim" -f"$CUT_COLS" "$file" > "${OUT_FILE}.tmp"
    mv "${OUT_FILE}.tmp" "$OUT_FILE"

    echo "-- Columns dropped successfully. Updated sample --"
    head -n 5 "$OUT_FILE" | column -ts "$delim"
    echo ""

    HEADER=$(head -1 "$OUT_FILE")
    IFS="$DELIM" read -ra COLS <<< "$HEADER"
else
    echo "! Cancelled column drop. !"
fi
```

```
    read -n1 -r -p "Press any key to continue..."
}
```

ฟังก์ชัน drop_columns() ใช้สำหรับ ลบคอลัมน์ที่ไม่ต้องการออกจากไฟล์ CSV โดยให้ผู้ใช้ระบุหมายเลขคอลัมน์ที่ต้องการลบ เช่น 1,3,5 แล้วระบบจะสร้างไฟล์ใหม่ที่มีเฉพาะคอลัมน์ที่เหลืออยู่

หลักการทำงานโดยรวม

1. รับค่าและตรวจสอบอินพุตจากผู้ใช้
    - ถ้า input = 0 หมายถึงไม่ลบคอลัมน์ใด ๆ ให้รันคำสั่ง cp "$file" "$OUT_FILE" เพื่อคัดลอกไฟล์เดิมไว้โดยไม่เปลี่ยนแปลง
    - ถ้าผู้ใช้กรอกไม่ถูกต้อง (ไม่อยู่ในรูปตัวเลขคั่นด้วย comma) จะตรวจด้วย grep -Eq '^[0-9]+(,[0-9]+)*$'
    - ถ้าไม่ตรงรูปแบบ จะขึ้นข้อความเตือนและจบการทำงาน
2. ตรวจสอบว่าหมายเลขคอลัมน์อยู่ในช่วงที่ถูกต้อง
    - ใช้ลูปตรวจสอบว่าตัวเลขที่ผู้ใช้กรอกอยู่ในช่วง 1–จำนวนคอลัมน์ทั้งหมด มิฉะนั้นจะเตือนว่า "out of range"
3. ยืนยันก่อนดำเนินการลบ
    - ใช้คำสั่ง read -p "Are you sure? (y/n): " cf เพื่อให้ผู้ใช้กดยืนยันก่อนดำเนินการจริง
4. สร้างรายการคอลัมน์ที่จะเก็บไว้ (ไม่ถูกลบ)
    - ใช้โครงสร้างอาเรย์ KEEP_ARR เก็บหมายเลขคอลัมน์ที่ไม่อยู่ในรายการลบ
5. ลบคอลัมน์ที่เลือกด้วยคำสั่ง cut
    - cut -d"$delim" -f"$CUT_COLS" "$file" > "${OUT_FILE}.tmp" cut -d ใช้กำหนดตัวคั่นคอลัมน์ เช่น , -f ระบุหมายเลขคอลัมน์ที่ต้องการเก็บไว้
    - จากนั้น mv ใช้ย้ายไฟล์ชั่วคราวไปทับไฟล์หลัก (อัปเดตข้อมูลใหม่) ด้วย mv "${OUT_FILE}.tmp" "$OUT_FILE"

6. แสดงตัวอย่างผลลัพธ์หลังการลบ
    - head -n 5 "$OUT_FILE" | column -ts "$delim" แสดง 5 แถวแรกของไฟล์ที่อัปเดตแล้ว จัดให้อยู่ในรูปแบบตารางให้อ่านง่าย
7. อัปเดตอาเรย์ชื่อคอลัมน์ (COLS)
8. อ่านชื่อคอลัมน์ใหม่จากไฟล์หลังลบเสร็จ เพื่อให้สอดคล้องกับข้อมูลล่าสุด

- **ฟังก์ชัน select_feature_types**

```
==============================
Choose an option: 3

Current column types:
1 - gender : Categorical
2 - age : Numeric
3 - hypertension : Categorical
4 - heart_disease : Categorical
5 - Marriage : Categorical
6 - work_type : Categorical
7 - Living_type : Categorical
8 - avg_glucose : Numeric
9 - bmi : Numeric
10 - smoking_status : Categorical
11 - illness : Categorical


Do you want to modify column types? (y/n): n
-- Keeping previous column type selection --
Press any key to continue...
```

```
select_feature_types() {

    local confirm="n"

    if [ ${#NUM_COLS[@]} -eq 0 ] && [ ${#CAT_COLS[@]} -eq 0 ]; then

        echo ""
```

```
echo "=== Specify column types for each feature ==="
echo "Enter 1 for Numeric  |   2 for Categorical"
echo ""


NUM_COLS=()
CAT_COLS=()


for id in "${!COLS[@]}"; do
    col="${COLS[$id]}"
    while true
    do
        read -p "Column $((id+1)) - $col : " ty
        if [ "$ty" = "1" ]; then
            NUM_COLS+=("$col")
            break
        elif [ "$ty" = "2" ]; then
            CAT_COLS+=("$col")
            break
        else
            echo "! Invalid input! Please enter 1 or 2 !"
        fi
```

```
        done
    done

    confirm="y"
    echo ""
    echo "-- Column types saved --"
    echo "-> Numeric columns: $(IFS=, ; echo "${NUM_COLS[*]}")"
    echo "-> Categorical columns: $(IFS=, ; echo "${CAT_COLS[*]}")"

else
    echo ""
    echo "Current column types:"
    for id in "${!COLS[@]}"; do
        col="${COLS[$id]}"
        type="Not set"
        for n in "${NUM_COLS[@]}"; do
            if [ "$n" = "$col" ]; then
                type="Numeric"
                break
            fi
        done
```

```
            for c in "${CAT_COLS[@]}"; do
                if [ "$c" = "$col" ]; then
                    type="Categorical"
                    break
                fi
            done
            echo "$((id+1)) - $col : $type"
        done
        echo ""
fi


while true
do
        echo ""
        read -p "Do you want to modify column types? (y/n): " modify
        if [ "$modify" = "y" ] || [ "$modify" = "Y" ]; then
            NUM_COLS=()
            CAT_COLS=()
            for id in "${!COLS[@]}"; do
                col="${COLS[$id]}"
                while true
```

```
            do
                echo ""
                read -p "Column $((id+1)) - $col : " ty
                if [ "$ty" = "1" ]; then
                    NUM_COLS+=("$col")
                    break
                elif [ "$ty" = "2" ]; then
                    CAT_COLS+=("$col")
                    break
                else
                    echo "! Please enter 1 or 2 !"
                fi
            done
        done

        echo ""
        echo "-- Column types updated --"
        echo "-> Numeric columns: $(IFS=, ; echo "${NUM_COLS[*]}")"
        echo "-> Categorical columns: $(IFS=, ; echo "${CAT_COLS[*]}")"

    elif [ "$modify" = "n" ] || [ "$modify" = "N" ]; then
```

```
            confirm="y"
            echo "-- Keeping previous column type selection --"
            break
        else
            echo "! Please enter y or n !"
        fi
    done
    read -n1 -r -p "Press any key to continue..."

}
```

กำหนดประเภทของแต่ละคอลัมม์

หลักการทำงานโดยรวม

1. ตรวจสอบว่ามีการกำหนดประเภทคอลัมน์แล้วหรือไม่
    - if [${#NUM_COLS[@]} -eq 0] && [${#CAT_COLS[@]} -eq 0]; then
    - ถ้ายังไม่มีการกำหนด จะให้ผู้ใช้กรอกประเภทคอลัมน์ทีละคอลัมน์
    - NUM_COLS และ CAT_COLS คืออาเรย์เก็บชื่อคอลัมน์ตัวเลขและเชิงหมวดหมู่
2. ระบุประเภทคอลัมน์
    - ใช้ ลูป for ผ่านอาเรย์ COLS (ชื่อคอลัมน์ทั้งหมด)
    - ใช้คำสั่ง read -p เพื่อรับค่าจากผู้ใช้ (1 = Numeric, 2 = Categorical)
    - เก็บลงในอาเรย์ที่เหมาะสม (NUM_COLS หรือ CAT_COLS)
3. แสดงคอลัมน์และประเภทที่บันทึกแล้ว

- echo "-> Numeric columns: $(IFS=, ; echo "${NUM_COLS[*]}")"
- echo "-> Categorical columns: $(IFS=, ; echo "${CAT_COLS[*]}")"
- ใช้ IFS กับ echo เพื่อรวมชื่อคอลัมน์เป็นข้อความคั่นด้วยเครื่องหมาย ,

4. แก้ไขประเภทคอลัมน์ถ้าต้องการ
    - ถามผู้ใช้ว่าอยากแก้ไขหรือไม่ (read -p "Do you want to modify column types? (y/n): ")
    - ถ้าแก้ไข จะล้างอาเรย์เดิมและให้ผู้ใช้ระบุประเภทใหม่อีกครั้ง
5. รอผู้ใช้ก่อนดำเนินการต่อ
    - read -n1 -r -p "Press any key to continue..." เพื่อให้ผู้ใช้ตรวจสอบและยืนยันก่อนออกจากฟังก์ชัน

- **ฟังก์ชั่น main_menu**

```
main_menu() {
    while true
    do
        echo ""
        echo "==============================="
        echo "      Data Cleaning Menu       "
        echo "==============================="
        echo "File: $OUT_FILE ($(cat $OUT_FILE | wc -l) rows, include column name)"
        echo ""
        echo "1) Show sample data"
        echo "2) View full data (press q to exit)"
```

```
echo "3) View / modify column types (Numeric / Categorical)"
echo "4) Check numeric columns"
echo "5) Handle missing values (Numeric / Categorical)"
echo "6) Handle outliers (Numeric only)"
echo "7) Normalize / scale numeric features"
echo "8) Check data validity / accuracy"
echo "9) Remove duplicate rows"
echo "10) Manual edit data"
echo "11) Drop column"
echo "12) Exit"
echo "============================="

read -p "Choose an option: " ch
case "$ch" in
    1)
        head -n 10 "$OUT_FILE" | column -ts "$DELIM"
        ;;
    2)
        cat -n "$OUT_FILE" | column -ts "$DELIM" | less
        ;;
    3)
```

select_feature_types

;;

4)

check_numeric_columns

;;

5)

handle_missing_values

;;

6)

handle_outliers

;;

7)

handle_normalize

;;

8)

check_data_validity

;;

9)

check_duplicates

;;

10)

```
                manual_edit_data
                ;;

            11)
                read -p "Enter column numbers to drop (comma separated) or 0 to keep all: " DROP_INPUT
                drop_columns "$DROP_INPUT"
                ;;
            12)
                echo "Exiting menu..."
                break
                ;;
            *)
                echo "! Please enter a number between 1-12 !"
                ;;
        esac
    done
}
```

ฟังก์ชัน main_menu() เป็นเมนูหลักสำหรับการทำ **Data Cleaning** ในไฟล์ CSV โดยให้ผู้ใช้เลือกขั้นตอนต่าง ๆ ของการตรวจสอบและปรับปรุงข้อมูล

หลักการทำงานโดยรวม

1. สร้างเมนูวนซ้ำ

    - ใช้ while true; do ... done เพื่อแสดงเมนูซ้ำ ๆ จนกว่าผู้ใช้จะเลือกออก (break)

2. แสดงข้อมูลไฟล์ปัจจุบัน

    - echo "File: $OUT_FILE ($(cat $OUT_FILE | wc -l) rows, include column name)"
    - cat $OUT_FILE | wc -l นับจำนวนแถวทั้งหมด
    - แสดงชื่อไฟล์และจำนวนแถวให้ผู้ใช้ทราบก่อนเลือกเมนู

3. แสดงตัวเลือกเมนู

    - เมนูมีตั้งแต่ดูตัวอย่างข้อมูล, ตรวจสอบ/แก้ไขประเภทคอลัมน์, จัดการค่าที่หายไป, ตรวจสอบ outlier, normalize, เช็คความถูกต้อง, ลบแถวซ้ำ, แก้ไขไฟล์ด้วยมือ, จนถึงลบคอลัมน์

4. รับตัวเลือกผู้ใช้

    - read -p "Choose an option: " ch
    - อ่านตัวเลขที่ผู้ใช้ป้อนเพื่อนำไปเลือกการทำงาน

5. สั่งงานตามตัวเลือก

    ใช้ case แยกแต่ละตัวเลือก เช่น

    - 1) แสดง 10 แถวตัวอย่าง
    - head -n 10 "$OUT_FILE" | column -ts "$DELIM"

```
==============================
Choose an option: 1
gender  age   hypertension  heart_disease  Marriage  work_type      Living_type  avg_glucose  bmi   smoking_status   illness
Male    67    0             1              Yes       Private        Urban        228.69       36.6  formerly smoked  1
Male    1000  0             1              Yes       Private        Rural        105.92       32.5  never smoked     1
Female  49    0             0              Yes       Private        Urban        171.5000     34.4  smokes           1
Female  79    1             0              Yes       Self-employed  Rural        174.12       24    never smoked     1
Male    81    0             0              Yes       Private        Urban        186.21       29    formerly smoked  1
Male    74    1             1              Yes       Private        Rural        70.09        27.4  never smoked     1
Female  69p   0             0              No        Private        Urban        94.39        22.8  never smoked     1
Female  78    0             0              Yes       Private        Urban        58.57        24.2  Unknown          1
Female  ff    1             0              Yes       Private        Rural        80.43        29.7  never smoked     1

==============================
      Data Cleaning Menu
==============================
File: xxx_dataset_clean.csv (401 rows, include column name)

1) Show sample data
2) View full data (press q to exit)
3) View / modify column types (Numeric / Categorical)
```

- 2) แสดง ข้อมูลทั้งหมด พร้อมหมายเลขแถว

	gender	age	hypertension	heart_disease	Marriage	work_type	Living_type	avg_glucose	bmi	smoking_status	il
2	Male	67	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
3	Male	1000	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
4	Female	49	0	0	Yes	Private	Urban	171.5000	34.4	smokes	1
5	Female	79	1	0	Yes	Self-employed	Rural	174.12	24	never smoked	1
6	Male	81	0	0	Yes	Private	Urban	186.21	29	formerly smoked	1
7	Male	74	1	1	Yes	Private	Rural	70.09	27.4	never smoked	1
8	Female	69p	0	0	No	Private	Urban	94.39	22.8	never smoked	1
9	Female	78	0	0	Yes	Private	Urban	58.57	24.2	Unknown	1
10	Female	ff	1	0	Yes	Private	Rural	80.43	29.7	never smoked	1
11	Female	61	0	1	Yes	Govt_job	Rural	120.46	36.8	smokes	1
12	Female	54	0	0	Yes	Private	Urban	104.51	27.3	smokes	1
13	Female	79	0	1	Yes	Private	Urban	214.09	28.2	never smoked	1
14	Female	50	1	0	Yes	Self-employed	Rural	167.41	30.9	never smoked	1
15	Male	64	0	1	Yes	Private	Urban	191.61	37.5	smokes	1
16	Male	75	1	0	Yes	Private	Urban	221.29	25.8	smokes	1
17	Female	60	0	0	No	Private	Urban	89.22	37.8	never smoked	1
18	Female	71	0	0	Yes	Govt_job	Rural	193.94	22.4	smokes	1
19	Female	52	1	0	Yes	Self-employed	Urban	233.29	48.9	never smoked	1
20	Female	79	0	0	Yes	Self-employed	Urban	228.7	26.6	never smoked	1
21	Male	82	0	1	Yes	Private	Rural	208.3	32.5	Unknown	1
22	Male	71	0	0	Yes	Private	Urban	102.87	27.2	formerly smoked	1
23	Male	80	0	0	Yes	Self-employed	Rural	104.12	23.5	never smoked	1
24	Female	65	0	0	Yes	Private	Rural	100.98	28.2	formerly smoked	1
25	Male	69	0	1	Yes	Self-employed	Urban	195.23	28.3	smokes	1
26	Male	57	1	0	Yes	Private	Urban	212.08	44.2	smokes	1
27	Male	42	0	0	Yes	Private	Rural	83.41	25.4	Unknown	1
28	Female	82	1	0	Yes	Self-employed	Urban	196.92	22.2	never smoked	1
29	Male	80	0	1	Yes	Self-employed	Urban	252.72	30.5	formerly smoked	1
30	Male	48	0	0	No	Govt_job	Urban	84.2	29.7	never smoked	1
31	Female	82	1	1	No	Private	Rural	84.03	26.5	formerly smoked	1
32	Male	74	0	0	Yes	Private	Rural	219.72	33.7	formerly smoked	1
33	Female	72	1	0	Yes	Private	Rural	74.63	23.1	formerly smoked	1
34	Male	58	0	0	No	Private	Rural	92.62	32	Unknown	1
35	Female	49	0	0	Yes	Private	Urban	60.91	29.9	never smoked	1
36	Male	78	0	0	Yes	Private	Rural	78.03	23.9	formerly smoked	1

:

- cat -n "$OUT_FILE" | column -ts "$DELIM" | less
  - cat -n เพิ่มหมายเลขแถว
  - column -ts "$DELIM" จัดให้อ่านง่ายเป็นตาราง
  - less แสดงหน้าจอแบบ scroll
- 3–11) เรียกฟังก์ชันอื่น ๆ ที่เกี่ยวกับการจัดการข้อมูล เช่น select_feature_types, check_numeric_columns, handle_missing_values, drop_columns
- 12) ออกจากเมนูโดยใช้ break

6. จัดการตัวเลือกไม่ถูกต้อง

- *) echo "! Please enter a number between 1-12 !"
- แจ้งผู้ใช้หากป้อนตัวเลขไม่อยู่ในช่วงที่กำหนด

3. ฟังก์ชันการทำงานใน main menu

- ฟังก์ชัน Check numeric columns

```
==============================
Choose an option: 4
=== Checking numeric columns for invalid data... ===

! Column age has invalid numeric values !
  Line 8 -> Invalid: "69p" | Row: Female,69p,0,0,No,Private,Urban,94.39,22.8,never smoked,1
  Line 10 -> Invalid: "ff" | Row: Female,ff,1,0,Yes,Private,Rural,80.43,29.7,never smoked,1

Choose how to fix invalid values in column ( age ):
1) Replace with mean
2) Replace with median
3) Replace with mode
4) Replace with custom value
5) Drop rows
6) Next column
7) Exit
Enter your choice: 3
-> Replacing with mode (80)
-> Fixed invalid values in age.

Press any key to continue...
```

```
check_numeric_columns() {

    local file="$OUT_FILE"

    local delim="$DELIM"

    local has_error=false
```

```
echo "=== Checking numeric columns for invalid data... ==="
echo ""
local header
header=$(head -n 1 "$file")
IFS="$delim" read -ra COLS <<< "$header"


for col_name in "${NUM_COLS[@]}"; do
	local col_num=0


	for i in "${!COLS[@]}"; do
		if [ "${COLS[$i]}" = "$col_name" ]; then
			col_num=$((i+1))
			break
		fi
	done
	local invalid_rows=()
	while IFS= read -r line;
	do
		invalid_rows+=("$line")
	done < <(
		awk -F"$delim" -v col="$col_num" '
```

```
        NR>1 && $col !~ /^ *-?[0-9]+(\.[0-9]+)? *$/ && $col !~ /^ *$/ {
        print NR ":" $col ":" $0 }
        ' "$file"
        )
if [ "${#invalid_rows[@]}" -gt 0 ]; then
        has_error=true
        echo "! Column $col_name has invalid numeric values !"
        for row in "${invalid_rows[@]}"; do
                IFS=':' read -r lineno val fullrow <<< "$row"
                echo "  Line $lineno -> Invalid: \"$val\" | Row: $fullrow"
        done


        echo ""
        echo "Choose how to fix invalid values in column ( $col_name ):"
        echo "1) Replace with mean"
        echo "2) Replace with median"
        echo "3) Replace with mode"
        echo "4) Replace with custom value"
        echo "5) Drop rows"
        echo "6) Next column"
        echo "7) Exit"
```

```
read -p "Enter your choice: " ch


while true
do
case $ch in
        1) mean_val=$(calc_mean "$file" "$col_num" "$delim")
                fill_val="$mean_val"
                echo "-> Replacing with mean ($fill_val)"
                break
                ;;


        2) median_val=$(calc_median "$file" "$col_num" "$delim")
                fill_val="$median_val"
                echo "-> Replacing with median ($fill_val)"
                break
                ;;
        3) mode_val=$(calc_mode "$file" "$col_num" "$delim")
                fill_val="$mode_val"
                echo "-> Replacing with mode ($fill_val)"
                break
```

```
        ;;
4) read -p "Enter custom value: " fill_val
        echo "-> Replacing with custom value ($fill_val)"
        break
        ;;
5)echo "-> Dropping rows with invalid values in $col_name..."
        for row in "${invalid_rows[@]}"; do
                IFS=':' read -r lineno val fullrow <<< "$row"
                sed -i "${lineno}d" "$file"
        done
        continue 2
        ;;


6)
        continue 2
        ;;
7)
        return
        ;;
*) echo "! Please input 1-7 !"
       continue
```

```
                        ;;
                esac
                done

                for row in "${invalid_rows[@]}"; do
                        IFS=':' read -r lineno val fullrow <<< "$row"
                        sed -i "${lineno}s/\(\(\([^$delim]*$delim\)\{$((col_num-1))\}\)
*\)[^$delim]*/\1$fill_val/" "$file"
                done
                echo "-> Fixed invalid values in $col_name."
                echo ""
            fi
        done

        if [ "$has_error" = false ]; then
            echo "-- All numeric columns are valid --"
        fi
        read -n1 -r -p "Press any key to continue..."
}
```

ฟังก์ชัน check_numeric_columns() ใช้สำหรับตรวจสอบและจัดการค่าที่ไม่ถูกต้องในคอลัมน์ตัวเลข (Numeric columns) ของไฟล์ CSV

หลักการทำงานโดยรวม

1. กำหนดไฟล์และตัวคั่น
    - local file="$OUT_FILE"
    - local delim="$DELIM"
    - ระบุไฟล์ CSV และตัวคั่นคอลัมน์เพื่อใช้ประมวลผล
2. อ่านชื่อคอลัมน์
    - header=$(head -n 1 "$file")
    - IFS="$delim" read -ra COLS <<< "$header"
    - ใช้ head -n 1 ดึงบรรทัดแรกเป็นชื่อคอลัมน
    - ใช้ IFS แยกชื่อคอลัมน์เป็นอาเรย์ COLS
3. ตรวจสอบค่าผิดรูปแบบในแต่ละคอลัมน์ตัวเลข
    - awk -F"$delim" -v col="$col_num" 'NR>1 && $col !~ /^ *-?[0-9]+(\.[0-9]+)? *$/ && $col !~ /^ *$/ {print NR ":" $col ":" $0}' "$file"
    - ตรวจสอบแต่ละแถว (NR>1)
    - ค่าที่ไม่ใช่ตัวเลข (รวมถึง float และ negative) หรือไม่ว่าง จะถูกเก็บเป็นแถวผิดปกติ
4. แสดงแถวที่มีค่าผิดปกติให้ผู้ใช้เลือกจัดการ
    - ผู้ใช้สามารถเลือก:
        - แทนค่าด้วยค่าเฉลี่ย (mean)
        - แทนค่าด้วยค่ากลาง (median)
        - แทนค่าด้วยค่าที่พบบ่อยที่สุด (mode)
        - แทนค่าด้วยค่ากำหนดเอง
        - ลบแถว
        - ข้ามคอลัมน์นี้
        - ออกจากฟังก์ชัน
5. ปรับค่าผิดปกติในไฟล์
    - ใช้ sed แก้ไขโดยตรงในไฟล์ (-i)

- sed -i "${lineno}s/\(\(\([^$delim]*$delim\)\{$((col_num-1))\}\)*\)[^$delim]*/\1$fill_val/" "$file"
- แทนค่าผิดปกติในคอลัมน์ที่ระบุด้วยค่าที่ผู้ใช้เลือก

6. แจ้งผลลัพธ์และรอผู้ใช้ก่อนดำเนินการต่อ
    - read -n1 -r -p "Press any key to continue..."

- **ฟังก์ชัน handle_missing_values**

```
==============================
Choose an option: 5

=== Handle Missing Values ===
Columns with missing values:
Column 1 - gender : 1 missing
Column 7 - Living_type : 1 missing
Column 9 - bmi : 1 missing

Do you want to view rows with missing values? (y/n): 1
! Please enter only y or n !
Do you want to view rows with missing values? (y/n): y
Enter column number to view missing rows: 1
Rows with missing values in column gender:
394:   76  0  0  Yes  Govt_job  Rural  84.21  24.4  never smoked  0

Do you want to view rows with missing values? (y/n): y
Enter column number to view missing rows: 9
Rows with missing values in column bmi:
379:Female  50  0  0  Yes  Private  Rural  93.47    never smoked  0

Do you want to view rows with missing values? (y/n): n
```

```
Do you want to view rows with missing values? (y/n): n

Column 1 - gender (Categorical) -> 1 missing
1) Fill with mode (Female)
2) Fill with 'Unknown'
3) Fill with custom value
4) Drop rows with missing
Enter your choice: 1
-> Missing values in gender replaced with 'Female'.

Column 7 - Living_type (Categorical) -> 1 missing
1) Fill with mode (Urban)
2) Fill with 'Unknown'
3) Fill with custom value
4) Drop rows with missing
Enter your choice: 2
-> Missing values in Living_type replaced with 'Unknown'.

Column 9 - bmi (Numeric) -> 1 missing
1) Fill with mean (29.511)
2) Fill with median (28.6)
3) Fill with mode (31.4)
4) Fill with custom value
5) Drop rows with missing
Enter your choice: 2
-> Missing values in bmi replaced with '28.6'.

-- Missing values handled successfully. --
```

```
handle_missing_values() {

    local file="$OUT_FILE"

    local delim="$DELIM"

    echo ""

    echo "=== Handle Missing Values ==="



    local missing_cols=()

    local missing_counts=()

    for id in "${!COLS[@]}"; do

        local col="${COLS[$id]}"

        local col_num=$((id+1))

        local MISSING_COUNT
```

```
        MISSING_COUNT=$(cut -d"$delim" -f"$col_num" "$file" | tail -n +2 | grep -c -E '^ *$')
        if [ "$MISSING_COUNT" -gt 0 ]; then
                missing_cols+=("$id")
                missing_counts+=("$MISSING_COUNT")
        fi
done

if [ "${#missing_cols[@]}" -eq 0 ]; then
        echo "-- No missing values found in any column. --"
        read -n1 -r -p "Press any key to return to menu..."
        return
fi

echo "Columns with missing values:"
for i in "${!missing_cols[@]}"; do
        idx="${missing_cols[$i]}"
        echo "Column $((idx+1)) - ${COLS[$idx]} : ${missing_counts[$i]} missing"
done
echo ""
```

```
while true
do
    read -p "Do you want to view rows with missing values? (y/n): " vi
    if [ "$vi" = "y" ] || [ "$vi" = "Y" ]; then
        read -p "Enter column number to view missing rows: " COL_VIEW
        is_missing_col=0
        for idx in "${missing_cols[@]}"; do
            if [ $((COL_VIEW-1)) -eq "$idx" ]; then
                is_missing_col=1
                break
            fi
        done
        if [ $is_missing_col -eq 0 ]; then
            echo "-> Column $COL_VIEW has no missing values!"
            continue
        fi
        echo "Rows with missing values in column ${COLS[$((COL_VIEW-1))]}:"
        grep -n -E "^([^$delim]*$delim){$((COL_VIEW-1))} *($delim|$)" "$file" | column -
ts"$delim"
        echo ""
    elif [ "$vi" = "n" ] || [ "$vi" = "N" ]; then
```

```
            break
        else
            echo "! Please enter only y or n !"
        fi
done




for id in "${missing_cols[@]}"; do
        local col="${COLS[$id]}"
        local col_num=$((id+1))
        local MISSING_COUNT
        MISSING_COUNT=$(cut -d"$delim" -f"$col_num" "$file" | tail -n +2 | grep -c -E '^ *$')


        local type="Unknown"
        type="Unknown"
        for n in "${NUM_COLS[@]}"; do
            if [ "$n" = "$col" ]; then
                type="Numeric"
            fi
        done
```

```
for c in "${CAT_COLS[@]}"; do
        if [ "$c" = "$col" ]; then
                type="Categorical"
        fi
done




echo ""
echo "Column $col_num - $col ($type) -> $MISSING_COUNT missing"


if [ "$type" = "Numeric" ]; then
        mean_val=$(calc_mean "$file" "$col_num" "$delim")
        median_val=$(calc_median "$file" "$col_num" "$delim")
        mode_val=$(calc_mode "$file" "$col_num" "$delim")


        echo "1) Fill with mean ($mean_val)"
        echo "2) Fill with median ($median_val)"
        echo "3) Fill with mode ($mode_val)"
        echo "4) Fill with custom value"
        echo "5) Drop rows with missing"
else
```

```
        mode_val=$(calc_mode "$file" "$col_num" "$delim")
        echo "1) Fill with mode ($mode_val)"
        echo "2) Fill with 'Unknown'"
        echo "3) Fill with custom value"
        echo "4) Drop rows with missing"
fi

read -p "Enter your choice: " ch
case $ch in
1)
        if [ "$type" = "Numeric" ]; then
                fill_val="$mean_val"
        else
                fill_val="$mode_val"
        fi
        ;;
2)
        if [ "$type" = "Numeric" ]; then
                fill_val="$median_val"
        else
                fill_val="Unknown"
```

```
            fi
            ;;
        3)
            read -p "Enter custom value to fill: " fill_val
            ;;

        4)
            if [ "$type" = "Numeric" ]; then
                read -p "Enter custom value to fill: " fill_val
                echo "-> Filling $col with custom value ($fill_val)"
            else
                echo "-> Dropping rows with missing in $col..."
                sed -E "/^(([^$delim]*$delim){$((col_num-1))}) *\$/d" "$file" >
"${OUT_FILE}.tmp"
                mv "${OUT_FILE}.tmp" "$OUT_FILE"
                break
            fi
            ;;

        5)
            if [ "$type" = "Numeric" ]; then
```

```
                        echo "-> Dropping rows with missing in $col..."
                        sed -E "/^(([^$delim]*$delim){$((col_num-1))}) *\$/d" "$file" >
"${OUT_FILE}.tmp"
                        mv "${OUT_FILE}.tmp" "$OUT_FILE"
                        break
                else
                        echo "Invalid choice!"
                        continue
                fi
                ;;

        *)
                echo "Invalid choice!"
                continue
                ;;
        esac

        sed -E "s/^(([^$delim]*$delim){$((col_num-1))}) *($delim|$)/\1$fill_val\3/" "$file" >
"${OUT_FILE}.tmp"
        mv "${OUT_FILE}.tmp" "$OUT_FILE"
        echo "-> Missing values in $col replaced with '$fill_val'."
    done
```

```
    echo ""
    echo "-- Missing values handled successfully. --"
    read -n1 -r -p "Press any key to continue..."
}
```

ฟังก์ชัน handle_missing_values() ใช้สำหรับตรวจสอบและจัดการค่าที่หายไป (missing values) ในไฟล์ CSV โดยให้ผู้ใช้เลือกวิธีจัดการแต่ละคอลัมน์ เช่น เติมค่าทดแทน หรือ ลบแถวที่มีค่าหายไป

หลักการทำงานโดยรวม

1. กำหนดไฟล์และตัวคั่น
    - local file="$OUT_FILE"
    - local delim="$DELIM"
    - ระบุไฟล์ CSV ที่ต้องจัดการและตัวคั่นคอลัมน์ (เช่น , หรือ ;)
2. ตรวจสอบคอลัมน์ที่มี missing values
    - ใช้คำสั่ง cut -d"$delim" -f"$col_num" "$file" ดึงคอลัมน์ที่สนใจ
    - tail -n +2 ข้ามหัวคอลัมน์
    - grep -c -E '^ *$' นับจำนวนแถวที่ว่างหรือไม่มีค่า
3. บันทึกคอลัมน์และจำนวน missing ไว้ในอาเรย์ missing_cols และ missing_counts
4. แสดงรายชื่อคอลัมน์ที่มีค่าหาย
    - ใช้ for loop แสดงชื่อคอลัมน์และจำนวนค่าหายให้ผู้ใช้เห็น
5. ให้ผู้ใช้ดูแถวที่มี missing values
    - ใช้ grep -n -E ตรวจจับแถวที่ค่าหายในคอลัมน์นั้น ๆ
    - แสดงผลด้วย column -ts"$delim" เพื่อให้อ่านง่าย
6. จัดการ missing values ตามชนิดของคอลัมน์

- ตรวจสอบว่าคอลัมน์เป็น Numeric หรือ Categorical
    - สำหรับ Numeric: ให้เลือกเติม mean, median, mode, ค่าที่กำหนดเอง หรือ ลบแถว
    - สำหรับ Categorical: ให้เลือกเติม mode, 'Unknown', ค่าที่กำหนดเอง หรือ ลบแถว
- ใช้ฟังก์ชัน calc_mean, calc_median, calc_mode สำหรับคำนวณค่าเติม

7. แก้ไขไฟล์ CSV
    - ใช้ sed -E เพื่อแทนค่า missing หรือ ลบแถวที่ตรงเงื่อนไข
    - บันทึกผลลงไฟล์ชั่วคราว ${OUT_FILE}.tmp แล้วใช้ mv แทนที่ไฟล์เดิม
8. หยุดรอผู้ใช้ก่อนดำเนินการต่อ
    - read -n1 -r -p "Press any key to continue..." ให้ผู้ใช้ตรวจสอบผลลัพธ์ก่อนกลับสู่เมนู

- **ฟังก์ชัน handle_outliers**

```
12) Exit
==============================
Choose an option: 6

=== Handle Outliers (Numeric only) ===

Available numeric columns:
1) age
2) avg_glucose
3) bmi
Select a column number: 1

Choose outlier detection method:
1) IQR method
2) Z-score method
Enter your choice (1 or 2): 1

-> Detecting outliers using IQR method...
Q1=44, Q3=75, IQR=31
Acceptable range: [-2.5, 121.5]

Do you want to remove outliers (y/n): y
-> Removing outliers in column age...
-- Outliers removed successfully --
Press any key to return to menu...
```

```
handle_outliers() {

    local file="$OUT_FILE"

    local delim="$DELIM"



    echo ""

    echo "=== Handle Outliers (Numeric only) ==="

    echo ""



    if [ ${#NUM_COLS[@]} -eq 0 ]; then

        echo "! No numeric columns found !"
```

```
        read -n1 -r -p "Press any key to return to menu..."
        return
fi

echo "Available numeric columns:"
for i in "${!NUM_COLS[@]}"; do
        echo "$((i+1))) ${NUM_COLS[$i]}"
done

read -p "Select a column number: " col_choice
local col_name="${NUM_COLS[$((col_choice-1))]}"
local col_num
col_num=$(get_col_index "$col_name")

if [ -z "$col_num" ]; then
        echo "Invalid column selection."
        read -n1 -r -p "Press any key to return..."
        return
fi

while true
```

```
do
    echo ""
    echo "Choose outlier detection method:"
    echo "1) IQR method"
    echo "2) Z-score method"
    read -p "Enter your choice (1 or 2): " met_ch

    if [ "$met_ch" = "1" ] || [ "$met_ch" = "2" ]; then
        break
    else
        echo "Invalid choice! Please enter 1 or 2."
    fi
done

case $met_ch in
    1)
        echo ""
        echo "-> Detecting outliers using IQR method..."
        vals=($(awk -F"$delim" -v c=$col_num 'NR>1 && $c !~ /^ *$/ {print $c}' "$file" |
sort -n))
        n=${#vals[@]}
```

```
        if [ $n -lt 4 ]; then
            echo "Not enough data to compute IQR."
            return
        fi

        q1_id=$(awk -v n=$n 'BEGIN{print int(0.25*(n-1))+1}')
        q3_id=$(awk -v n=$n 'BEGIN{print int(0.75*(n-1))+1}')
        Q1=${vals[$((q1_id-1))]}
        Q3=${vals[$((q3_id-1))]}
        IQR=$(awk -v q1=$Q1 -v q3=$Q3 'BEGIN{print q3-q1}')
        LOWER=$(awk -v q1=$Q1 -v iqr=$IQR 'BEGIN{print q1 - 1.5*iqr}')
        UPPER=$(awk -v q3=$Q3 -v iqr=$IQR 'BEGIN{print q3 + 1.5*iqr}')
        echo "Q1=$Q1, Q3=$Q3, IQR=$IQR"
        echo "Acceptable range: [$LOWER, $UPPER]"
        ;;
    2)
        echo ""
        echo "-> Detecting outliers using Z-score method..."
        mean_val=$(calc_mean "$file" "$col_num" "$delim")
        std_val=$(calc_std "$file" "$col_num" "$delim")
        echo "Mean=$mean_val, Std=$std_val"
```

```
                ;;
        esac


        echo ""
        if confirm_input "Do you want to remove outliers"; then
                echo "-> Removing outliers in column $col_name..."
                if [ "$met_ch" -eq 1 ]; then
                        awk -F"$delim" -v c=$col_num -v low="$LOWER" -v up="$UPPER" -v
OFS="$delim" \
                        'NR==1 || ($c >= low && $c <= up)' "$file" > "${OUT_FILE}.tmp"
                else
                        awk -F"$delim" -v c=$col_num -v mean="$mean_val" -v std="$std_val" -v
OFS="$delim" \
                        'NR==1 || (std>0 && (($c-mean)/std <= 3 && ($c-mean)/std >= -3))' "$file" >
"${OUT_FILE}.tmp"
                fi
                mv "${OUT_FILE}.tmp" "$OUT_FILE"
                echo "-- Outliers removed successfully --"
        else
                echo "Outlier removal cancelled"
        fi
```

```
    read -n1 -r -p "Press any key to return to menu..."
}
```

ฟังก์ชัน handle_outliers() ใช้สำหรับตรวจจับและจัดการค่าผิดปกติ (outliers) ในคอลัมน์เชิงตัวเลขของไฟล์ CSV

หลักการทำงานโดยรวม

1. กำหนดไฟล์และตัวคั่น
    - local file="$OUT_FILE"
    - local delim="$DELIM"
    - กำหนดไฟล์ CSV และตัวคั่นคอลัมน์ที่ใช้ในการประมวลผล
2. ตรวจสอบว่ามีคอลัมน์ตัวเลขหรือไม่
    - หากไม่มีคอลัมน์ตัวเลข (NUM_COLS) จะหยุดฟังก์ชัน และรอให้ผู้ใช้กดปุ่มเพื่อกลับเมนู
3. เลือกคอลัมน์ตัวเลขที่ต้องการจัดการ outliers
4. แสดงรายการคอลัมน์ตัวเลข
    - ให้ผู้ใช้เลือกหมายเลขคอลัมน์
    - ใช้ฟังก์ชัน get_col_index เพื่อหา index ของคอลัมน์
5. เลือกวิธีตรวจจับ outliers
    - ผู้ใช้สามารถเลือกได้ระหว่าง
        - IQR method (Interquartile Range)
        - Z-score method
6. ตรวจจับ outliers
    - IQR method

        - vals=($(awk -F"$delim" -v c=$col_num 'NR>1 && $c !~ /^ *$/ {print $c}' "$file" | sort -n))
        - awk ดึงค่าจากคอลัมน์ที่เลือก (ยกเว้นบรรทัดหัวข้อ)
        - sort -n เรียงค่าตัวเลข
        - คำนวณ Q1, Q3 และ IQR เพื่อตั้งขอบเขตค่าที่ยอมรับได้
    - Z-score method
        - ใช้ฟังก์ชัน calc_mean และ calc_std เพื่อคำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน
        - ค่า Z-score เกิน ±3 จะถือเป็น outlier
7. ยืนยันการลบ outliers
    - ใช้ฟังก์ชัน confirm_input เพื่อให้ผู้ใช้ตัดสินใจลบ outliers หรือไม่
8. ลบค่า outliers และบันทึกผล
    - ใช้ awk กรองข้อมูลให้เหลือเฉพาะค่าที่อยู่ในช่วงที่ยอมรับได้ (IQR) หรือ Z-score ≤ 3
    - บันทึกลงไฟล์ชั่วคราว ${OUT_FILE}.tmp แล้วใช้ mv "${OUT_FILE}.tmp" "$OUT_FILE" เพื่อแทนไฟล์เดิม
9. หยุดรอผู้ใช้ก่อนกลับเมนู

- **ฟังก์ชัน handle_normalize**

```
12) Exit
==============================
Choose an option: 7

=== Normalize / Scale Numeric Features ===

Available numeric columns:
1) age
2) avg_glucose
3) bmi
Select a column number to normalize/scale: 1

Choose normalization/scaling method:
1) Min-Max scaling (0-1)
2) Z-score standardization
Enter your choice: 1

Normalizing/scaling column 'age'...
This may take a few seconds, please wait...
-- Column 'age' normalized/scaled successfully --
Press any key to return to menu...
```

```
============================
      Data Cleaning Menu
============================
File: xxx_dataset_clean.csv (400 rows, include column name)

1) Show sample data
2) View full data (press q to exit)
3) View / modify column types (Numeric / Categorical)
4) Check numeric columns
5) Handle missing values (Numeric / Categorical)
6) Handle outliers (Numeric only)
7) Normalize / scale numeric features
8) Check data validity / accuracy
9) Remove duplicate rows
10) Manual edit data
11) Drop column
12) Exit
============================
Choose an option:
gender  age       hypertension  heart_disease  Marriage  work_type      Living_type  avg_glucose  bmi   smoking_status   illness
Male    0.815271  0             1              Yes       Private        Urban        228.69       36.6  formerly smoked  1
Female  0.593596  0             0              Yes       Private        Urban        171.5000     34.4  smokes           1
Female  0.963054  1             0              Yes       Self-employed  Rural        174.12       24    never smoked     1
Male    0.987685  0             0              Yes       Private        Urban        186.21       29    formerly smoked  1
Male    0.901478  1             1              Yes       Private        Rural        70.09        27.4  never smoked     1
Female  0.975369  0             0              No        Private        Urban        94.39        22.8  never smoked     1
Female  0.950739  0             0              Yes       Private        Urban        58.57        24.2  Unknown          1
Female  0.975369  1             0              Yes       Private        Rural        80.43        29.7  never smoked     1
Female  0.741379  0             1              Yes       Govt_job       Rural        120.46       36.8  smokes           1

============================
```

```
handle_normalize() {

    local file="$OUT_FILE"

    local delim="$DELIM"
```

```
echo ""
echo "=== Normalize / Scale Numeric Features ==="
echo ""

if [ ${#NUM_COLS[@]} -eq 0 ]; then
    echo "! No numeric columns found !"
    read -n1 -r -p "Press any key to return to menu..."
    return
fi

echo "Available numeric columns:"
for i in "${!NUM_COLS[@]}"; do
    echo "$((i+1))) ${NUM_COLS[$i]}"
done

local col_ch

while true
do
```

```
        read -p "Select a column number to normalize/scale: " col_ch
        if [[ "$col_ch" =~ ^[0-9]+$ ]] && [ "$col_ch" -ge 1 ] && [ "$col_ch" -le
"${#NUM_COLS[@]}" ]; then
            break
        else
            echo "! Please enter a valid number !"
        fi
    done

    local col_name="${NUM_COLS[$((col_ch-1))]}"
    local col_num
    col_num=$(get_col_index "$col_name")

    if [ -z "$col_num" ]; then
        echo "Invalid column index."
        return
    fi


    echo ""
    echo "Choose normalization/scaling method: "
```

```
echo "1) Min-Max scaling (0-1)"

echo "2) Z-score standardization"

local met_ch


while true

do

	read -p "Enter your choice: " met_ch

	if [ "$met_ch" = "1" ] || [ "$met_ch" = "2" ]; then

		break

	else

		echo "Invalid choice! Enter 1 or 2."

	fi

done


col_val=($(tail -n +2 "$file" | cut -d"$delim" -f"$col_num" | grep -v -E '^ *$'))


local min_val max_val mean_val std_val

if [ "$met_ch" == "1" ]; then

	min_val=$(printf "%s\n" "${col_val[@]}" | sort -n | head -n1)

	max_val=$(printf "%s\n" "${col_val[@]}" | sort -n | tail -n1)
```

```
else
	mean_val=$(calc_mean "$file" "$col_num" "$delim")
	std_val=$(calc_std "$file" "$col_num" "$delim")
fi


head -n1 "$file" > "${OUT_FILE}.tmp"


echo ""
echo "Normalizing/scaling column '$col_name'..."
echo "This may take a few seconds, please wait..."


while IFS="$delim" read -r line; do
	val=$(echo "$line" | cut -d"$delim" -f"$col_num")


	if echo "$val" | grep -q -E '^ *$'; then
		new_line="$line"
	else
		if [ "$met_ch" = "1" ]; then
			norm_val=$(awk -v x="$val" -v min="$min_val" -v max="$max_val" \
			'BEGIN{printf "%.6f", (max-min==0)?0:(x-min)/(max-min)}')
		else
```

```
                    norm_val=$(awk -v x="$val" -v mean="$mean_val" -v std="$std_val" \
                    'BEGIN{printf "%.6f", (std==0)?0:(x-mean)/std}')
                fi
                new_line=$(echo "$line" | sed -E "s/^(([^$delim]*$delim){$((col_num-
1))})[^$delim]+/\1$norm_val/")
            fi
            echo "$new_line" >> "${OUT_FILE}.tmp"
        done < <(tail -n +2 "$file")



        mv "${OUT_FILE}.tmp" "$OUT_FILE"
        echo "-- Column '$col_name' normalized/scaled successfully --"
        read -n1 -r -p "Press any key to return to menu..."
}
```

ฟังก์ชัน handle_normalize() ใช้สำหรับ ปรับมาตรฐาน (normalize/scale) ค่าตัวเลขของคอลัมน์ในไฟล์ CSV ให้เป็นช่วงหรือสเกลที่เหมาะสม เช่น Min-Max (0–1) หรือ Z-score

หลักการทำงานโดยรวม

1. กำหนดไฟล์และตัวคั่น
    - local file="$OUT_FILE"
    - local delim="$DELIM"
    - เพื่อระบุไฟล์ CSV และตัวคั่นคอลัมน์ (เช่น , หรือ ;)

2. ตรวจสอบว่ามีคอลัมน์ตัวเลขหรือไม่
    - หากไม่มีคอลัมน์ตัวเลข (NUM_COLS) ฟังก์ชันจะหยุดและรอผู้ใช้กดปุ่ม
3. แสดงคอลัมน์ตัวเลขและให้ผู้ใช้เลือกคอลัมน์
    - ใช้ for เพื่อแสดงชื่อคอลัมน์ตัวเลขทั้งหมด
    - รับตัวเลขของคอลัมน์ที่ผู้ใช้เลือก (read) และตรวจสอบว่าเป็นเลขที่ถูกต้อง
4. เลือกวิธีปรับสเกล
    - Min-Max scaling: (x - min) / (max - min)
    - Z-score standardization: (x - mean) / std
5. ใช้ read เพื่อรับตัวเลือกจากผู้ใช้
6. อ่านค่าจากคอลัมน์ที่เลือก
    - col_val=($(tail -n +2 "$file" | cut -d"$delim" -f"$col_num" | grep -v -E '^ *$'))
    - tail -n +2 ข้ามหัวคอลัมน์
    - cut -d"$delim" -f"$col_num" ดึงค่าของคอลัมน์ที่เลือก
    - grep -v -E '^ *$' ลบค่าเว้นว่าง
7. คำนวณค่า min/max หรือ mean/std
    - Min-Max: ใช้ sort -n กับ head/tail หา min/max
    - Z-score: ใช้ฟังก์ชัน calc_mean และ calc_std
8. ปรับค่าคอลัมน์ทีละแถวและบันทึกชั่วคราว
    - while IFS="$delim" read -r line; do

      ...

      echo "$new_line" >> "${OUT_FILE}.tmp"

      done < <(tail -n +2 "$file")
    - อ่านทีละแถวของไฟล์ (เว้นหัวคอลัมน์)
    - แทนค่าของคอลัมน์ที่เลือกด้วยค่าที่ปรับสเกลแล้ว (awk และ sed)
    - เขียนลงไฟล์ชั่วคราว ${OUT_FILE}.tmp

- แทนที่ไฟล์เดิมด้วยไฟล์ชั่วคราว mv "${OUT_FILE}.tmp" "$OUT_FILE" เพื่อให้การเปลี่ยนแปลงมีผลถาวร

9. รอผู้ใช้ก่อนกลับสู่เมนู

- **ฟังก์ชัน check_data_validity**

```
==============================
Choose an option: 8

=== Check Data Validity / Accuracy ===


-> Checking numeric columns...

Column: age
Min: 0.000000, Max: 1.000000
Do you want to filter values outside a specific range? (y/n): u
! Invalid input. Please enter only y or n !
Do you want to filter values outside a specific range? (y/n): n

Column: avg_glucose
Min: 56.07, Max: 271.74
Do you want to filter values outside a specific range? (y/n): y
Enter minimum allowed value: 60
Enter maximum allowed value: 270
-> Rows outside [60, 270] removed.

Column: bmi
Min: 0.000000, Max: 1.000000
Do you want to filter values outside a specific range? (y/n): n
```

```
----------------------------------------

-> Checking categorical columns...

Column: gender
208  Female
181  Male
Do you want to replace invalid/mistyped values in gender? (y/n): n

Column: hypertension
320  0
69   1
Do you want to replace invalid/mistyped values in hypertension? (y/n): n
^[[A^[[D^[[Cnn
Column: heart_disease
339  0
50   1
Do you want to replace invalid/mistyped values in heart_disease? (y/n): n
! Invalid input. Please enter only y or n !
Do you want to replace invalid/mistyped values in heart_disease? (y/n): n

Column: Marriage
297  Yes
92   No
Do you want to replace invalid/mistyped values in Marriage? (y/n): y
Enter value to replace (current): Yes
Enter new value: Y
-> Replaced 'Yes' with 'Y' in column Marriage
Replace another value in Marriage? (y/n): y
Enter value to replace (current): No
Enter new value: n
-> Replaced 'No' with 'n' in column Marriage
Replace another value in Marriage? (y/n): n

Column: work_type
222  Private
82   Self-employed
49   Govt_job
36   children
Do you want to replace invalid/mistyped values in work_type? (y/n):
```

```
check_data_validity() {

    local file="$OUT_FILE"

    local delim="$DELIM"



    echo ""

    echo "=== Check Data Validity / Accuracy ==="

    echo ""

    if [ ${#NUM_COLS[@]} -gt 0 ]; then
```

```
echo ""
echo "-> Checking numeric columns..."
for col_name in "${NUM_COLS[@]}"; do
    col_num=$(get_col_index "$col_name")
    echo ""
    echo "Column: $col_name"

    min_val=$(awk -F"$delim" -v c=$col_num 'NR>1 && $c !~ /^ *$/ {if(min=="" ||
$c<min) min=$c} END{print min}' "$file")
    max_val=$(awk -F"$delim" -v c=$col_num 'NR>1 && $c !~ /^ *$/ {if(max=="" ||
$c>max) max=$c} END{print max}' "$file")
    echo "Min: $min_val, Max: $max_val"

    if confirm_input "Do you want to filter values outside a specific range?"; then
        read -p "Enter minimum allowed value: " min_allow
        read -p "Enter maximum allowed value: " max_allow
        awk -F"$delim" -v c=$col_num -v min=$min_allow -v max=$max_allow -v
OFS="$delim" \
        'NR==1 || ($c !~ /^ *$/ && $c >= min && $c <= max)' "$file" >
"${OUT_FILE}.tmp"
        mv "${OUT_FILE}.tmp" "$OUT_FILE"
        echo "-> Rows outside [$min_allow, $max_allow] removed."
```

```
                fi

            done
        fi


        if [ ${#CAT_COLS[@]} -gt 0 ]; then
            echo ""
            echo "----------------------------------------"
            echo ""
            echo "-> Checking categorical columns..."
            for col_name in "${CAT_COLS[@]}"; do
                col_num=$(get_col_index "$col_name")
                echo ""
                echo "Column: $col_name"
                cut -d"$delim" -f"$col_num" "$file" | tail -n +2 | sort | uniq -c | sort -nr | column
-t

                if confirm_input "Do you want to replace invalid/mistyped values in
$col_name?"; then
                    while true
                    do
```

```
            while true
            do
                read -p "Enter value to replace (current): " old_val
                exists=$(awk -F"$delim" -v c=$col_num -v val="$old_val" \
                    'NR>1 && $c==val {found=1} END{print found+0}' "$file")
                if [ "$exists" -eq 1 ]; then
                    break
                else
                    echo "! Value '$old_val' not found in column
$col_name. Please enter a valid existing value."
                fi
            done


            read -p "Enter new value: " new_val
            sed -i "s/$old_val/$new_val/g" "$file"
            echo "-> Replaced '$old_val' with '$new_val' in column $col_name"


            if ! confirm_input "Replace another value in $col_name?"; then
                break
            fi
        done
```

```
                fi

            done
        fi

        echo ""
        read -n1 -r -p "Press any key to return to menu..."

}
```

ฟังก์ชัน check_data_validity() ใช้สำหรับตรวจสอบความถูกต้องและความสมเหตุสมผลของข้อมูลในไฟล์ CSV ทั้งคอลัมน์ตัวเลขและคอลัมน์หมวดหมู่ เพื่อให้ผู้ใช้สามารถกรองค่าไม่ถูกต้องหรือแก้ไขค่าผิดพลาดได้ก่อนการประมวลผลต่อไป

หลักการทำงานโดยรวม

1. กำหนดไฟล์และตัวคั่นคอลัมน์
    - local file="$OUT_FILE"
    - local delim="$DELIM"
    - ใช้ไฟล์ผลลัพธ์ที่ผ่านการทำความสะอาดแล้ว และตัวคั่นคอลัมน์จากขั้นก่อน
2. ตรวจสอบคอลัมน์ตัวเลข (Numeric Columns)
    - วนลูปแต่ละคอลัมน์ตัวเลข
    - ใช้ awk หาค่าต่ำสุด (min_val) และค่าสูงสุด (max_val) ของแต่ละคอลัมน์

- awk -F"$delim" -v c=$col_num 'NR>1 && $c !~ /^ *$/ {if(min=="" || $c<min) min=$c} END{print min}' "$file"
- NR>1 ข้ามบรรทัดหัวตาราง
- $c !~ /^ *$/ ข้ามช่องว่าง
    - ถ้าผู้ใช้ต้องการกำหนดช่วงค่า (min_allow, max_allow) ใช้ awk กรองเฉพาะแถวที่อยู่ในช่วงนั้นและเขียนลงไฟล์ใหม่โดยใช้ mv "${OUT_FILE}.tmp" "$OUT_FILE" เพื่อแทนที่ไฟล์เดิม

3. ตรวจสอบคอลัมน์หมวดหมู่ (Categorical Columns)
    - ใช้ cut -d"$delim" -f"$col_num" ดึงคอลัมน์ที่สนใจ
    - ใช้ tail -n +2 | sort | uniq -c | sort -nr | column -t แสดงรายการค่าที่พบ พร้อมจำนวนครั้งการเกิด
    - ถ้าผู้ใช้ต้องการแก้ไขค่าใช้ awk ตรวจสอบว่าค่าที่ต้องการแก้มีจริง จากนั้นใช้ sed -i แทนค่าทันที
4. หยุดรอผู้ใช้ก่อนกลับเมนู
    - read -n1 -r -p "Press any key to return to menu..."

- **ฟังก์ชัน check_duplicates**

```
==============================
      Data Cleaning Menu
==============================
File: xxx_dataset_clean.csv (390 rows, include column name)

1) Show sample data
2) View full data (press q to exit)
3) View / modify column types (Numeric / Categorical)
4) Check numeric columns
5) Handle missing values (Numeric / Categorical)
6) Handle outliers (Numeric only)
7) Normalize / scale numeric features
8) Check data validity / accuracy
9) Remove duplicate rows
10) Manual edit data
11) Drop column
12) Exit
==============================
Choose an option: 9

=== Checking for duplicate rows ===
Found 1 duplicate rows.
Male  0.568966  0  0  Y  Private  Rural  93.55  0.474474  never smoked  0
Do you want to remove duplicate rows? (y/n): y
Duplicate rows removed.

==============================
      Data Cleaning Menu
==============================
File: xxx_dataset_clean.csv (389 rows, include column name)

1) Show sample data
2) View full data (press q to exit)
3) View / modify column types (Numeric / Categorical)
4) Check numeric columns
5) Handle missing values (Numeric / Categorical)
6) Handle outliers (Numeric only)
7) Normalize / scale numeric features
8) Check data validity / accuracy
9) Remove duplicate rows
10) Manual edit data
11) Drop column
12) Exit
```

```
check_duplicates() {

    local file="$OUT_FILE"

    local delim="$DELIM"



    echo ""

    echo "=== Checking for duplicate rows ==="
```

```
HEADER=$(head -1 "$file")

DATA=$(tail -n +2 "$file")


DUP_COUNT=$(echo "$DATA" | sort | uniq -d | wc -l)


if [ "$DUP_COUNT" -eq 0 ]; then

        echo "No duplicate rows found."

        return

else

        echo "Found $DUP_COUNT duplicate rows."

        echo "$DATA" | sort | uniq -d | column -ts "$delim"

fi


while true

do

        read -p "Do you want to remove duplicate rows? (y/n): " CONFIRM_DUP

        if [ "$CONFIRM_DUP" = "y" ] || [ "$CONFIRM_DUP" = "Y" ]; then

                echo "$HEADER" > "${OUT_FILE}.tmp"

                echo "$DATA" | sort | uniq >> "${OUT_FILE}.tmp"

                mv "${OUT_FILE}.tmp" "$OUT_FILE"

                echo "Duplicate rows removed."
```

```
            break
        elif [ "$CONFIRM_DUP" = "n" ] || [ "$CONFIRM_DUP" = "N" ]; then
            echo "Duplicate removal cancelled."
            break
        else
            echo "! Please enter y or n !"
        fi
    done
}
```

ฟังก์ชัน check_duplicates() ใช้สำหรับตรวจสอบและลบแถวซ้ำในไฟล์ CSV เพื่อให้ข้อมูลสะอาดก่อนการวิเคราะห์ต่อไป

หลักการทำงานโดยรวม

1. กำหนดไฟล์และตัวคั่น
    - local file="$OUT_FILE"
    - local delim="$DELIM"
    - เพื่อระบุไฟล์ CSV และตัวคั่นคอลัมน์ที่ใช้อยู่
2. แยกชื่อคอลัมน์และข้อมูลจริง
    - HEADER=$(head -1 "$file")
    - DATA=$(tail -n +2 "$file"
    - head -1 ดึงบรรทัดแรกของไฟล์ เป็นชื่อคอลัมน์
    - tail -n +2 ดึงข้อมูลตั้งแต่บรรทัดที่ 2 เป็นต้นไป
3. ตรวจสอบแถวซ้ำ

- DUP_COUNT=$(echo "$DATA" | sort | uniq -d | wc -l)
- sort เรียงข้อมูลให้ซ้ำกันอยู่ติดกัน
- uniq -d แสดงเฉพาะแถวที่ซ้ำ
- wc -l นับจำนวนแถวซ้ำ

4. แสดงผล
    - ถ้าไม่มีแถวซ้ำ แจ้งว่า "No duplicate rows found."
    - ถ้ามีแถวซ้ำ แสดงจำนวนและตัวอย่าง โดยใช้ column -ts "$delim" เพื่อจัดรูปแบบให้อ่านง่าย
5. ให้ผู้ใช้เลือกว่าจะลบแถวซ้ำหรือไม่
    - ถ้าเลือก "y"
        - echo "$HEADER" > "${OUT_FILE}.tmp"
        - echo "$DATA" | sort | uniq >> "${OUT_FILE}.tmp"
        - mv "${OUT_FILE}.tmp" "$OUT_FILE"
        - สร้างไฟล์ชั่วคราวเก็บชื่อคอลัมน์
        - เพิ่มข้อมูลไม่ซ้ำ
        - ใช้ mv แทนที่ไฟล์เดิม
    - ถ้าเลือก "n" จะคงข้อมูลเดิม

- **ฟังก์ชัน manual_edit_data**

```
=== Manual Data Editing ===

Displaying data (header included) ...
```

1	gender	age	hypertension	heart_disease	Marriage	work_type	Living_type	avg_glucose	bmi	smoking_status	illness
2	Female	0.005419	0	0	n	children	Urban	62.4	0.195195	Unknown	0
3	Female	0.014778	0	0	n	children	Rural	109.56	0.024024	Unknown	0
4	Female	0.027094	0	0	n	children	Rural	94.12	0.174174	Unknown	0
5	Female	0.027094	0	0	n	children	Rural	97.6	0.306306	Unknown	0
6	Female	0.027094	0	0	n	children	Urban	82.91	0.129129	Unknown	0
7	Female	0.051724	0	0	n	children	Rural	77.83	0.000000	Unknown	0
8	Female	0.051724	0	0	n	children	Urban	84.91	0.315315	Unknown	0
9	Female	0.064039	0	0	n	children	Rural	119.88	0.066066	Unknown	0
10	Female	0.113300	0	0	n	children	Urban	112.34	0.075075	Unknown	0
11	Female	0.125616	0	0	n	children	Urban	130.15	0.048048	Unknown	0
12	Female	0.125616	0	0	n	children	Urban	73.66	0.147147	never smoked	0
13	Female	0.137931	0	0	n	children	Urban	86.55	0.327327	Unknown	0
14	Female	0.150246	0	0	n	children	Rural	102.27	0.048048	never smoked	0
15	Female	0.150246	0	0	n	children	Rural	78.38	0.693694	Unknown	0
16	Female	0.162562	0	0	n	children	Urban	91.85	0.366366	never smoked	0
17	Female	0.162562	0	0	n	Private	Rural	118.81	0.273273	Unknown	0
18	Female	0.199507	0	0	n	Private	Rural	112.23	0.393393	never smoked	0
19	Female	0.199507	0	0	n	Private	Urban	70.01	0.822823	Unknown	0
20	Female	0.248768	0	0	n	Private	Rural	112.38	0.306306	Unknown	0
21	Female	0.248768	0	0	n	Private	Urban	134.45	0.405405	never smoked	0
22	Female	0.248768	0	0	n	Private	Urban	90.16	0.399399	smokes	0
23	Female	0.261084	0	0	n	Private	Urban	63.37	0.327327	never smoked	0
24	Female	0.285714	0	0	Y	Private	Urban	89.68	0.693694	never smoked	0
25	Female	0.298030	0	0	n	Private	Rural	92.82	0.255255	never smoked	0
26	Female	0.298030	0	0	n	Private	Urban	82.77	0.621622	Unknown	0
27	Female	0.310345	0	0	n	Private	Rural	89.3	0.984985	smokes	0
28	Female	0.310345	0	0	Y	Private	Rural	91.88	0.279279	formerly smoked	0
29	Female	0.334975	0	0	Y	Private	Rural	141.15	0.390390	never smoked	0
30	Female	0.347291	0	0	n	Private	Urban	72.02	0.552553	formerly smoked	0
31	Female	0.359606	0	0	n	Private	Urban	96.42	0.210210	Unknown	0
32	Female	0.359606	0	0	Y	Govt_job	Rural	62.25	0.543544	never smoked	0
33	Female	0.384236	0	0	n	Self-employed	Rural	93.17	0.357357	smokes	0
34	Female	0.384236	0	0	Y	Private	Rural	76.13	0.429429	smokes	1
35	Female	0.396552	0	0	Y	Govt_job	Rural	76.66	0.276276	never smoked	0
36	Female	0.421182	0	0	Y	Self-employed	Urban	70.87	0.195195	formerly smoked	0
37	Female	0.445813	0	0	Y	Private	Rural	156.7	0.639640	never smoked	0
38	Female	0.445813	0	0	Y	Private	Rural	98.12	0.357357	never smoked	0
39	Female	0.445813	0	0	Y	Private	Urban	70.75	0.606607	Unknown	0
40	Female	0.458128	0	0	n	Self-employed	Urban	82.28	0.252252	formerly smoked	1
41	Female	0.458128	0	0	Y	Private	Urban	69.34	0.843844	never smoked	0
42	Female	0.470443	0	0	Y	Private	Urban	83.24	0.321321	never smoked	1
43	Female	0.470443	0	0	Y	Self-employed	Urban	97.76	0.420420	smokes	1
44	Female	0.482759	0	0	n	Private	Urban	78.04	0.504505	smokes	0
45	Female	0.482759	0	0	n	Private	Urban	80.47	0.351351	smokes	0
46	Female	0.482759	0	0	Y	Private	Urban	103.09	0.600601	Unknown	0
47	Female	0.495074	0	0	Y	Govt_job	Rural	229.86	0.588589	smokes	0

:

```
Do you want to edit a value? (y/n): y
Enter the row number to edit (or 'g' to replace a value in all rows of a column): g
Enter the column number to edit: 3
Enter value to search for in column 3: 0
Enter new value to replace with: No
Replace ALL occurrences of '0' with 'No' in column 3? (y/n): y
-- All occurrences in column 3 replaced --

Do you want to edit a value? (y/n): n
-- Manual editing finished --
Press any key to continue...

==============================
      Data Cleaning Menu
==============================
File: xxx_dataset_clean.csv (389 rows, include column name)

1) Show sample data
2) View full data (press q to exit)
3) View / modify column types (Numeric / Categorical)
4) Check numeric columns
5) Handle missing values (Numeric / Categorical)
6) Handle outliers (Numeric only)
7) Normalize / scale numeric features
8) Check data validity / accuracy
9) Remove duplicate rows
10) Manual edit data
11) Drop column
12) Exit
==============
Choose an option: 1
gender  age       hypertension  heart_disease  Marriage  work_type  Living_type  avg_glucose  bmi       smoking_status  illness
Female  0.005319  No            0              n         children   Urban        62.4         0.195195  Unknown         0
Female  0.014778  No            0              n         children   Rural        109.56       0.024024  Unknown         0
Female  0.027094  No            0              n         children   Rural        94.12        0.174174  Unknown         0
Female  0.027094  No            0              n         children   Rural        97.6         0.306306  Unknown         0
Female  0.027094  No            0              n         children   Urban        82.91        0.129129  Unknown         0
Female  0.051724  No            0              n         children   Rural        77.83        0.000000  Unknown         0
Female  0.051724  No            0              n         children   Urban        84.91        0.315315  Unknown         0
Female  0.064039  No            0              n         children   Rural        119.88       0.066066  Unknown         0
Female  0.113300  No            0              n         children   Urban        112.34       0.075075  Unknown         0

==============================
```

```
manual_edit_data() {

    local file="$OUT_FILE"

    local delim="$DELIM"



    echo ""

    echo "=== Manual Data Editing ==="

    echo ""



    echo "Displaying data (header included) ..."

    echo ""

    cat -n "$file" | column -ts "$DELIM" | less
```

```
echo ""

while confirm_input "Do you want to edit a value?"
do
    read -p "Enter the row number to edit (or 'g' to replace a value in all rows of a column): " row_input
    read -p "Enter the column number to edit: " col_num

    if [ "$row_input" = "g" ]; then

        while true
        do
            read -p "Enter value to search for in column $col_num: " old_val
            exists=$(awk -F"$delim" -v c=$col_num -v val="$old_val" 'NR>1 && $c==val {found=1} END{print found+0}' "$file")
            if [ "$exists" -eq 1 ]; then
                break
            else
                echo "! Value '$old_val' not found in column $col_num. Please enter a valid existing value."
            fi
        done
```

```
                read -p "Enter new value to replace with: " new_val

                if confirm_input "Replace ALL occurrences of '$old_val' with '$new_val' in
column $col_num?"; then
                        sed -E -i "s/^(([^$delim]*$delim){$((col_num-1))})${old_val}/\1$new_val/"
"$file"
                        echo "-- All occurrences in column $col_num replaced --"
                        echo ""
                else
                        echo "! Skipped !"
                fi
                continue
        fi

        row_num="$row_input"
        old_val=$(awk -F"$delim" -v r="$row_num" -v c="$col_num" 'NR==r {print $c}' "$file")
        echo "Current value at row $row_num, column $col_num: '$old_val'"

        read -p "Enter new value: " new_val

        if confirm_input "Are you sure you want to replace '$old_val' with '$new_val'?"; then
```

```
                sed -E -i "${row_num}s/^(([^$delim]*$delim){$((col_num-
1))})[^$delim]*/\1$new_val/" "$file"
                echo "-- Row $row_num, Column $col_num updated to '$new_val' --"
            else
                echo "! Skipped !"
            fi

            echo ""
        done

        echo "-- Manual editing finished --"
        read -n1 -r -p "Press any key to continue..."
}
```

ฟังก์ชัน manual_edit_data() ใช้สำหรับให้ผู้ใช้แก้ไขข้อมูลในไฟล์ CSV ด้วยตนเอง ทั้งแบบแก้ไขค่าเฉพาะแถวหรือแทนค่าทั้งคอลัมน์

หลักการทำงานโดยรวม

1. กำหนดไฟล์และตัวคั่นที่ใช้
    - local file="$OUT_FILE"
    - local delim="$DELIM"
    - กำหนดไฟล์ CSV และตัวคั่นคอลัมน์ (DELIM) เพื่อใช้ในการอ่านและแก้ไขข้อมูล
2. แสดงข้อมูลทั้งหมดให้ผู้ใช้ดู

- cat -n "$file" | column -ts "$delim" | less
- cat -n แสดงเนื้อหาพร้อมหมายเลขแถว
- column -ts "$delim" จัดรูปแบบเป็นตารางตามตัวคั่น
- less ให้ผู้ใช้เลื่อนดูข้อมูลได้สะดวก

3. ลูปให้ผู้ใช้แก้ไขค่าตามต้องการ
    - ใช้ฟังก์ชัน confirm_input เพื่อถามผู้ใช้ว่าต้องการแก้ไขหรือไม่
    - ผู้ใช้สามารถเลือกแก้ไข เฉพาะแถว หรือ แทนค่าทั้งคอลัมน์ (g)
4. แก้ไขค่าในแถวหรือคอลัมน์
    - ดึงค่าเดิมจากไฟล์ด้วย awk
    - awk -F"$delim" -v r="$row_num" -v c="$col_num" 'NR==r {print $c}' "$file"
    - แก้ไขค่าด้วย sed -E -i
    - sed -E -i "${row_num}s/^(([^$delim]*$delim){$((col_num-1))})[^$delim]*/\1$new_val/" "$file"
    - ^(([^$delim]*$delim){n}) ใช้จับถึงคอลัมน์ที่ต้องการ
    - แทนค่าเดิมด้วย $new_val
        - สำหรับแทนค่าทั้งคอลัมน์ จะตรวจสอบก่อนว่าค่าที่ผู้ใช้จะเปลี่ยนมีอยู่จริงในคอลัมน์นั้นด้วย awk
5. ยืนยันก่อนแก้ไขทุกครั้ง
    - ผู้ใช้ต้องตอบยืนยันก่อนที่จะบันทึกการแก้ไข
6. รอผู้ใช้ก่อนออกจากฟังก์ชัน
    - read -n1 -r -p "Press any key to continue..."

4. ฟังก์ชันอื่น ๆ

- **ฟังก์ชัน calc_std**

```
calc_std() {
    local file="$1"
    local col="$2"
    local delim="$3"


    awk -F"$delim" -v c="$col" '
    NR > 1 && $c ~ /^ *-?[0-9]+(\.[0-9]+)? *$/ {
        sum += $c
        sumsq += ($c) * ($c)
        n++
    }
    END {
        if (n > 1) {
            var = (sumsq - (sum*sum)/n) / (n-1)
            printf "%.6f\n", sqrt(var)
        } else {
            print 0
        }
    }' "$file"
```

}

ใช้คำนวณ ค่าเบี่ยงเบนมาตรฐาน (Standard Deviation) ของคอลัมน์ตัวเลขในไฟล์ CSV

หลักการทำงาน

1. ใช้ awk -F"$delim" เพื่อแยกคอลัมน์ตามตัวคั่น ($delim)
2. ข้ามบรรทัดแรก (NR > 1) และตรวจสอบว่าค่าเป็นตัวเลข ($c ~ /^ *-?[0-9]+(\.[0-9]+)? *$/)
3. สะสมผลรวม (sum) และผลรวมกำลังสอง (sumsq)
4. คำนวณค่าเบี่ยงเบนมาตรฐาน

$$S.D. = \sqrt{\frac{\sum(x_i - \overline{x})^2}{(n-1)}}$$

5. แสดงผลแบบทศนิยม 6 ตำแหน่ง

- **ฟังก์ชัน calc_mean**

```
calc_mean() {
    local file="$1"
    local col="$2"
    local delim="$3"
    tail -n +2 "$file" | awk -F"$delim" -v c="$col" '($c !~ /^ *$/) {
    sum += $c
    count++
    } END {
```

```
        if (count > 0)
                printf "%.3f\n", sum / count
        else
                print "NaN"
        }'
}
```

ใช้คำนวณ ค่าเฉลี่ย (Mean) ของคอลัมน์ตัวเลข

หลักการทำงาน

1. ข้ามบรรทัดแรก (tail -n +2) เพื่อไม่เอา header
2. ใช้ awk -F"$delim" อ่านแต่ละค่าและสะสมผลรวม (sum) และนับจำนวนค่า (count)
3. คำนวณค่าเฉลี่ย sum / count และแสดงผลแบบทศนิยม 3 ตำแหน่ง

- **ฟังก์ชัน calc_median**

```
calc_median() {
        local file="$1"
        local col="$2"
        local delim="$3"
        tail -n +2 "$file" | awk -F"$delim" -v c="$col" '$c !~ /^ *$/ {print $c}' | sort -n | awk '{
        data[NR] = $1
```

```
        } END {
        if (NR == 0) {
                print 0; exit
        }
        mid = int((NR + 1) / 2)
        if (NR % 2 == 1)
                print data[mid]
        else
                print (data[mid] + data[mid + 1]) / 2
        }'
}
```

ใช้คำนวณ ค่ามัธยฐาน (Median) ของคอลัมน์ตัวเลข

หลักการทำงาน

1. ข้าม header (tail -n +2)
2. ใช้ awk ดึงค่าที่ไม่ว่าง ($c !~ /^ *$/)
3. เรียงลำดับค่าด้วย sort -
4. ใช้ awk เก็บค่าในอาเรย์ data[NR] แล้วหาค่ากลาง
    - ถ้าแถวจำนวนคี่ = ค่ากลางตรง
    - ถ้าแถวจำนวนคู่ = ค่าเฉลี่ยของสองค่ากลาง

- **ฟังก์ชัน calc_std**

```
calc_mode() {
	local file="$1"
	local col="$2"
	local delim="$3"
	tail -n +2 "$file" | awk -F"$delim" -v c="$col" '$c !~ /^ *$/ {count[$c]++} END {
	max = 0
	for (v in count) {
		if (count[v] > max) {
			max = count[v]
			mode = v
		}
	}
	print mode
	}'
}
```

ใช้หาค่า โหมด (Mode) หรือค่าที่เกิดซ้ำมากที่สุดในคอลัมน์

หลักการทำงาน

1. ข้าม header (tail -n +2)

2. ใช้ awk นับจำนวนแต่ละค่า (count[$c]++)
3. หา key ที่มีค่าสูงสุด (max) และพิมพ์ค่าโหมด

- **ฟังก์ชัน confirm_input**

```
confirm_input() {
    local prompt="$1"
    local ans
    while true; do
        read -p "$prompt (y/n): " ans
        if [ "$ans" = "y" ] || [ "$ans" = "Y" ]; then
            return 0
        elif [ "$ans" = "n" ] || [ "$ans" = "N" ]; then
            return 1
        else
            echo "! Invalid input. Please enter only y or n !"
        fi
    done
}
```

หลักการทำงานโดยรวม

1. รับข้อความ Prompt จากพารามิเตอร์
    - local prompt="$1"
    - กำหนดข้อความที่จะใช้ถามผู้ใช้ เช่น “Do you want to continue?”
2. ลูปรับคำตอบจากผู้ใช้
    - while true; do

      read -p "$prompt (y/n): " ans

      ...

      done
    - ใช้ read -p เพื่อแสดงข้อความและรับข้อมูลจากผู้ใช้
    - ลูป while true ทำงานต่อเนื่องจนกว่าผู้ใช้จะกรอกคำตอบถูกต้อง
3. ตรวจสอบคำตอบของผู้ใช้

```
if [ "$ans" = "y" ] || [ "$ans" = "Y" ]; then
return 0
elif [ "$ans" = "n" ] || [ "$ans" = "N" ]; then
return 1
else
echo "! Invalid input. Please enter only y or n !"
fi
```

    - ถ้าผู้ใช้กรอก y หรือ Y คืนค่า 0 (มักหมายถึง “ตกลง” ใน Bash
    - ถ้าผู้ใช้กรอก n หรือ N คืนค่า 1 (หมายถึง “ไม่ตกลง”)
    - หากกรอกค่าอื่น แจ้งข้อผิดพลาดและวนลูปถามใหม่

อ้างอิง

ข้อมูล Dataset ที่ใช้ในการทดสอบ

gender,age,hypertension,heart_disease,Marriage,work_type,Living_type,avg_glucose,bmi,smoking_status,illness

Male,67,0,1,Yes,Private,Urban,228.69,36.6,formerly smoked,1

Male,1000,0,1,Yes,Private,Rural,105.92,32.5,never smoked,1

Female,49,0,0,Yes,Private,Urban,171.5000,34.4,smokes,1

Female,79,1,0,Yes,Self-employed,Rural,174.12,24,never smoked,1

Male,81,0,0,Yes,Private,Urban,186.21,29,formerly smoked,1

Male,74,1,1,Yes,Private,Rural,70.09,27.4,never smoked,1

Female,69p,0,0,No,Private,Urban,94.39,22.8,never smoked,1

Female,78,0,0,Yes,Private,Urban,58.57,24.2,Unknown,1

Female,ff,1,0,Yes,Private,Rural,80.43,29.7,never smoked,1

Female,61,0,1,Yes,Govt_job,Rural,120.46,36.8,smokes,1

Female,54,0,0,Yes,Private,Urban,104.51,27.3,smokes,1

Female,79,0,1,Yes,Private,Urban,214.09,28.2,never smoked,1

Female,50,1,0,Yes,Self-employed,Rural,167.41,30.9,never smoked,1

Male,64,0,1,Yes,Private,Urban,191.61,37.5,smokes,1

Male,75,1,0,Yes,Private,Urban,221.29,25.8,smokes,1

Female,60,0,0,No,Private,Urban,89.22,37.8,never smoked,1

Female,71,0,0,Yes,Govt_job,Rural,193.94,22.4,smokes,1

Female,52,1,0,Yes,Self-employed,Urban,233.29,48.9,never smoked,1

Female,79,0,0,Yes,Self-employed,Urban,228.7,26.6,never smoked,1

Male,82,0,1,Yes,Private,Rural,208.3,32.5,Unknown,1

Male,71,0,0,Yes,Private,Urban,102.87,27.2,formerly smoked,1

Male,80,0,0,Yes,Self-employed,Rural,104.12,23.5,never smoked,1

Female,65,0,0,Yes,Private,Rural,100.98,28.2,formerly smoked,1

Male,69,0,1,Yes,Self-employed,Urban,195.23,28.3,smokes,1

Male,57,1,0,Yes,Private,Urban,212.08,44.2,smokes,1

Male,42,0,0,Yes,Private,Rural,83.41,25.4,Unknown,1

Female,82,1,0,Yes,Self-employed,Urban,196.92,22.2,never smoked,1

Male,80,0,1,Yes,Self-employed,Urban,252.72,30.5,formerly smoked,1

Male,48,0,0,No,Govt_job,Urban,84.2,29.7,never smoked,1

Female,82,1,1,No,Private,Rural,84.03,26.5,formerly smoked,1

Male,74,0,0,Yes,Private,Rural,219.72,33.7,formerly smoked,1

Female,72,1,0,Yes,Private,Rural,74.63,23.1,formerly smoked,1

Male,58,0,0,No,Private,Rural,92.62,32,Unknown,1

Female,49,0,0,Yes,Private,Urban,60.91,29.9,never smoked,1

Male,78,0,0,Yes,Private,Rural,78.03,23.9,formerly smoked,1

Male,54,0,0,Yes,Private,Urban,71.22,28.5,never smoked,1

Male,82,0,1,Yes,Private,Urban,144.9,26.4,smokes,1

Male,60,1,0,Yes,Govt_job,Urban,213.03,20.2,smokes,1

Male,76,1,0,Yes,Private,Rural,243.58,33.6,never smoked,1

Female,58,0,0,Yes,Private,Urban,107.26,38.6,formerly smoked,1

Male,81,0,0,Yes,Self-employed,Urban,99.33,33.7,never smoked,1

Female,39,1,0,Yes,Private,Rural,58.09,39.2,smokes,1

Female,79,0,1,Yes,Private,Rural,127.29,27.7,never smoked,1

Female,77,1,0,Yes,Self-employed,Urban,124.13,31.4,never smoked,1

Male,63,0,1,Yes,Private,Rural,196.71,36.5,formerly smoked,1

Female,82,0,0,Yes,Private,Rural,59.32,33.2,never smoked,1

Male,73,1,0,Yes,Self-employed,Urban,194.99,32.8,never smoked,1

Female,54,1,0,Yes,Govt_job,Urban,180.93,27.7,never smoked,1

Female,56,0,0,Yes,Private,Urban,185.17,40.4,formerly smoked,1

Female,80,1,0,Yes,Private,Rural,74.9,22.2,never smoked,1

Female,67,1,0,Yes,Self-employed,Rural,61.94,25.3,smokes,1

Female,45,0,0,Yes,Private,Rural,93.72,30.2,formerly smoked,1

Male,78,1,0,No,Private,Urban,113.01,24,never smoked,1

Female,70,0,0,Yes,Private,Urban,221.58,47.5,never smoked,1

Male,76,0,0,Yes,Private,Urban,104.47,20.3,Unknown,1

Male,59,0,0,Yes,Private,Urban,86.23,30,formerly smoked,1

Female,80,1,0,Yes,Self-employed,Rural,72.67,28.9,never smoked,1

Female,67,1,0,Yes,Private,Rural,179.12,28.1,formerly smoked,1

Female,66,1,0,Yes,Govt_job,Rural,116.55,31.1,formerly smoked,1

Male,63,0,0,Yes,Private,Rural,228.56,27.4,never smoked,1

Female,52,0,0,Yes,Private,Rural,96.59,26.4,never smoked,1

Female,80,0,1,Yes,Self-employed,Rural,66.72,21.7,formerly smoked,1

Male,80,1,0,Yes,Private,Urban,240.09,27,never smoked,1

Female,79,0,0,Yes,Private,Urban,110.85,24.1,formerly smoked,1

Male,43,0,0,Yes,Self-employed,Urban,143.43,45.9,Unknown,1

Male,59,0,0,Yes,Private,Rural,96.16,44.1,Unknown,1

Female,79,0,0,No,Private,Rural,88.92,22.9,never smoked,1

Female,68,1,0,No,Self-employed,Urban,79.79,29.7,never smoked,1

Male,54,0,0,Yes,Private,Rural,96.97,29.1,smokes,1

Male,61,0,1,Yes,Private,Rural,111.81,27.3,smokes,1

Female,70,0,1,Yes,Private,Rural,59.35,32.3,formerly smoked,1

Male,47,0,0,Yes,Private,Urban,86.94,41.1,formerly smoked,1

Male,74,0,0,Yes,Private,Urban,98.55,25.6,Unknown,1

Female,79,0,1,Yes,Private,Urban,226.98,29.8,never smoked,1

Male,81,0,0,Yes,Private,Urban,72.81,26.3,never smoked,1

Female,57,0,0,No,Self-employed,Urban,68.02,37.5,never smoked,1

Female,80,1,0,Yes,Self-employed,Urban,68.56,26.2,Unknown,1

Male,45,0,0,Yes,Private,Rural,64.14,29.4,never smoked,1

Female,78,0,0,Yes,Private,Rural,235.63,32.3,never smoked,1

Female,70,0,0,Yes,Self-employed,Rural,76.34,24.4,formerly smoked,1

Male,58,0,1,Yes,Private,Rural,240.59,31.4,smokes,1

Male,57,1,0,Yes,Govt_job,Urban,78.92,27.7,formerly smoked,1

Female,69,0,0,Yes,Govt_job,Urban,82.81,28,never smoked,1

Male,64,0,1,Yes,Govt_job,Urban,74.1,28.8,Unknown,1

Female,77,0,0,Yes,Govt_job,Rural,190.32,31.4,never smoked,1

Female,74,0,0,Yes,Private,Rural,231.61,34.6,formerly smoked,1

Female,81,0,1,Yes,Self-employed,Rural,78.7,19.4,Unknown,1

Female,57,0,0,Yes,Govt_job,Urban,110.52,28.5,Unknown,1

Female,50,1,0,Yes,Private,Rural,73.18,30.3,formerly smoked,1

Female,54,0,0,Yes,Self-employed,Rural,191.82,40.4,smokes,1

Female,79,0,0,Yes,Private,Rural,93.05,24.2,never smoked,1

Female,53,0,0,Yes,Govt_job,Urban,64.17,41.5,never smoked,1

Male,79,0,1,Yes,Private,Rural,129.98,22.6,formerly smoked,1

Male,80,0,1,Yes,Private,Urban,68.53,24.2,smokes,1

Female,68,0,0,No,Govt_job,Urban,82.1,27.1,Unknown,1

Male,71,1,1,No,Govt_job,Rural,216.94,30.9,never smoked,1

Male,61,1,0,Yes,Self-employed,Rural,76.11,27.3,smokes,1

Male,74,0,0,Yes,Private,Urban,72.96,31.3,smokes,1

Female,38,0,0,No,Self-employed,Urban,82.28,24,formerly smoked,1

Female,77,0,0,Yes,Private,Urban,105.22,31,never smoked,1

Female,58,0,0,Yes,Private,Rural,59.86,28,formerly smoked,1

Female,53,1,0,Yes,Private,Urban,62.55,30.3,Unknown,1

Male,80,0,0,Yes,Private,Rural,259.63,31.7,smokes,1

Male,56,1,0,Yes,Private,Rural,249.31,35.8,never smoked,1

Female,72,0,0,Yes,Govt_job,Urban,131.41,28.4,never smoked,1

Female,80,0,0,Yes,Private,Urban,73.54,24,Unknown,1

Male,82,0,0,Yes,Govt_job,Urban,200.59,29,formerly smoked,1

Female,73,1,0,Yes,Private,Rural,190.14,36.5,never smoked,1

Female,78,1,0,No,Private,Urban,130.54,20.1,never smoked,1

Male,69,1,0,No,Private,Rural,182.99,36.5,never smoked,1

Female,68,1,0,Yes,Self-employed,Rural,206.09,26.7,never smoked,1

Female,71,0,0,Yes,Govt_job,Urban,263.32,38.7,never smoked,1

Male,76,0,0,Yes,Private,Urban,140.1,29.9,formerly smoked,1

Female,76,0,0,Yes,Private,Urban,207.28,34.9,Unknown,1

Male,76,1,0,Yes,Self-employed,Rural,194.37,27,formerly smoked,1

Female,75,0,0,Yes,Private,Rural,199.2,26.6,Unknown,1

Male,82,0,1,Yes,Govt_job,Rural,103.68,25,never smoked,1

Female,59,0,0,Yes,Private,Rural,116.44,23.8,smokes,1

Female,74,1,0,Yes,Private,Urban,70.28,21.8,never smoked,1

Female,69,1,1,No,Self-employed,Urban,72.17,36.8,never smoked,1

Female,72,1,0,Yes,Private,Urban,221.79,30,never smoked,1

Male,66,0,0,Yes,Private,Urban,151.16,27.5,formerly smoked,1

Female,78,0,0,Yes,Private,Urban,67.29,24.6,never smoked,1

Female,57,1,0,Yes,Govt_job,Rural,67.41,32.9,never smoked,1

Female,70,0,1,Yes,Private,Rural,239.07,26.1,never smoked,1

Male,68,0,1,Yes,Private,Urban,223.83,31.9,formerly smoked,1

Female,80,0,0,Yes,Self-employed,Urban,76.57,34.1,never smoked,1

Male,68,0,0,Yes,Self-employed,Urban,77.82,27.5,smokes,1

Female,55,0,0,Yes,Self-employed,Rural,92.98,25.6,never smoked,1

Female,77,0,1,Yes,Private,Rural,231.56,36.9,never smoked,1

Female,50,0,0,Yes,Private,Rural,102.16,31.4,smokes,1

Female,57,0,0,Yes,Private,Urban,221.89,37.3,smokes,1

Female,71,0,0,Yes,Self-employed,Urban,195.71,34.1,formerly smoked,1

Female,81,1,0,Yes,Self-employed,Rural,74.02,25,never smoked,1

Female,78,1,0,No,Private,Urban,203.87,45.7,never smoked,1

Female,70,1,0,Yes,Self-employed,Urban,89.13,34.2,formerly smoked,1

Male,78,0,0,Yes,Self-employed,Urban,133.19,23.6,formerly smoked,1

Female,56,0,0,Yes,Govt_job,Rural,162.23,27.3,Unknown,1

Female,79,1,0,No,Govt_job,Urban,98.02,22.3,formerly smoked,1

Male,81,0,0,Yes,Self-employed,Rural,91.54,31.4,never smoked,1

Female,60,0,0,Yes,Private,Urban,97.43,26.4,smokes,1

Female,80,1,0,Yes,Self-employed,Rural,91.02,32.9,formerly smoked,1

Male,72,1,0,Yes,Self-employed,Rural,185.49,37.1,never smoked,1

Female,80,0,0,Yes,Private,Urban,64.44,45,never smoked,1

Male,78,1,0,Yes,Self-employed,Rural,56.11,25.5,formerly smoked,1

Male,81,0,0,Yes,Private,Urban,213.22,26.1,Unknown,1

Female,46,0,0,Yes,Private,Urban,78.18,30.8,never smoked,1

Male,59,0,0,Yes,Private,Urban,111.04,32,formerly smoked,1

Female,32,0,0,Yes,Private,Rural,76.13,29.9,smokes,1

Male,61,1,1,Yes,Private,Urban,112.24,37.4,smokes,1

Male,63,0,0,Yes,Private,Rural,137.3,31.7,formerly smoked,1

Male,63,0,1,Yes,Self-employed,Urban,116.69,34.5,formerly smoked,1

Female,82,1,1,Yes,Govt_job,Urban,215.94,27.9,formerly smoked,1

Male,82,0,0,Yes,Private,Rural,86.62,29.5,formerly smoked,1

Female,65,0,0,Yes,Private,Urban,205.77,46,formerly smoked,1

Male,51,1,0,Yes,Private,Urban,112.16,42.5,formerly smoked,1

Male,59,0,0,Yes,Self-employed,Rural,118.03,35.5,smokes,1

Male,68,1,1,Yes,Private,Rural,271.74,31.1,smokes,1

Female,72,0,0,Yes,Private,Rural,97.92,26.9,smokes,1

Male,59,0,0,Yes,Private,Urban,200.62,35.8,formerly smoked,1

Male,70,1,0,Yes,Private,Rural,242.52,45.5,formerly smoked,1

Male,69,0,0,Yes,Private,Rural,93.81,28.5,Unknown,1

Female,74,0,0,Yes,Self-employed,Urban,74.96,26.6,never smoked,1

Male,80,1,1,Yes,Private,Urban,175.29,31.5,formerly smoked,1

Female,77,0,0,Yes,Self-employed,Rural,90,32,never smoked,1

Female,78,0,0,Yes,Self-employed,Rural,109.47,30.8,never smoked,1

Male,54,0,0,Yes,Govt_job,Rural,87.85,31.1,smokes,1

Female,78,0,0,Yes,Private,Urban,106.74,33,formerly smoked,1

Female,81,0,0,Yes,Private,Rural,80.13,23.4,never smoked,1

Male,78,0,0,No,Self-employed,Urban,90.19,26.9,never smoked,1

Male,78,1,0,Yes,Self-employed,Rural,134.8,33.6,Unknown,1

Female,63,0,0,Yes,Govt_job,Rural,106.58,23.9,Unknown,1

Female,39,0,0,Yes,Private,Urban,83.24,26.3,never smoked,1

Female,51,0,0,Yes,Govt_job,Rural,103.43,27.3,formerly smoked,1

Male,63,0,0,Yes,Private,Urban,208.65,30.7,never smoked,1

Female,48,0,0,Yes,Private,Urban,74.11,20.5,never smoked,1

Female,81,0,0,Yes,Self-employed,Urban,95.84,21.5,never smoked,1

Female,79,0,1,No,Private,Urban,205.33,31,smokes,1

Male,78,0,0,Yes,Self-employed,Urban,116.1,27.1,never smoked,1

Female,55,1,1,Yes,Private,Urban,210.4,40,smokes,1

Male,73,0,1,Yes,Govt_job,Rural,219.73,28.6,never smoked,1

Male,81,1,1,Yes,Private,Urban,250.89,28.1,smokes,1

Male,79,0,0,Yes,Private,Rural,72.73,28.4,never smoked,1

Female,63,0,0,Yes,Govt_job,Rural,205.35,42.2,formerly smoked,1

Female,81,0,0,No,Govt_job,Urban,70.3,25.8,smokes,1

Male,49,0,0,No,Private,Rural,104.86,31.9,smokes,1

Female,57,0,1,Yes,Private,Urban,216.58,31,Unknown,1

Female,81,0,0,Yes,Private,Rural,184.4,27.5,never smoked,1

Female,39,0,0,Yes,Self-employed,Urban,97.76,29.6,smokes,1

Female,80,1,0,No,Private,Urban,66.03,35.4,never smoked,1

Female,81,0,0,Yes,Self-employed,Rural,81.95,16.9,never smoked,1

Female,79,0,0,Yes,Private,Urban,97.73,21.5,smokes,1

Female,73,0,0,Yes,Self-employed,Urban,70.94,34.4,never smoked,1

Female,77,1,0,Yes,Self-employed,Urban,199.84,28,formerly smoked,1

Male,78,0,0,Yes,Self-employed,Urban,218.46,26.8,Unknown,1

Female,68,0,0,Yes,Private,Rural,211.06,39.3,Unknown,1

Male,57,0,0,Yes,Private,Urban,86.3,31.7,Unknown,1

Male,77,0,0,Yes,Self-employed,Rural,162.14,32.6,formerly smoked,1

Female,51,1,0,Yes,Private,Urban,88.2,28.4,never smoked,1

Male,60,0,1,Yes,Private,Urban,91.92,35.9,smokes,1

Male,46,0,0,Yes,Self-employed,Urban,83.12,29.6,formerly smoked,0

Male,53,0,0,Yes,Self-employed,Urban,94.89,28.5,never smoked,0

Female,79,0,0,No,Private,Rural,239.52,25.5,never smoked,0

Male,43,0,0,Yes,Self-employed,Rural,99.15,30.5,formerly smoked,0

Female,56,0,0,Yes,Self-employed,Urban,124.16,23,never smoked,0

Male,3,0,0,No,children,Rural,86.38,22.8,Unknown,0

Male,32,0,0,No,Private,Rural,93.68,31.4,never smoked,0

Female,41,0,0,Yes,Govt_job,Rural,229.86,35.2,smokes,0

Male,53,0,0,Yes,Govt_job,Rural,78.73,23.3,never smoked,0

Male,67,0,0,Yes,Govt_job,Rural,93.71,31.2,formerly smoked,0

Female,33,0,0,Yes,Govt_job,Rural,76.66,24.8,never smoked,0

Female,21,0,0,No,Private,Rural,112.38,25.8,Unknown,0

Female,26,0,0,No,Private,Rural,89.3,48.4,smokes,0

Female,69,1,0,Yes,Govt_job,Rural,103.44,43.1,formerly smoked,0

Male,15,0,0,No,Private,Rural,61.61,27.8,never smoked,0

Female,3,0,0,No,children,Rural,94.12,21.4,Unknown,0

Female,1.24,0,0,No,children,Urban,62.4,22.1,Unknown,0

Female,80,1,0,Yes,Private,Urban,210.96,31.8,never smoked,0

Male,2,0,0,No,children,Urban,76.25,20.1,Unknown,0

Male,50,0,0,Yes,Private,Urban,103.48,29.1,smokes,0

Female,30,0,0,No,Private,Urban,96.42,22.6,Unknown,0

Male,15,0,0,No,Private,Rural,78.59,25.1,Unknown,0

Female,71,1,0,Yes,Private,Rural,195.25,33.3,never smoked,0

Male,4,0,0,No,children,Rural,103.76,15.9,Unknown,0

Male,2,0,0,No,children,Rural,68.52,20.8,Unknown,0

Male,11,0,0,No,children,Urban,92.17,19.5,Unknown,0

Female,45,0,0,Yes,Private,Rural,81.02,39,never smoked,0

Male,22,0,0,No,Govt_job,Rural,94.33,23.1,never smoked,0

Male,63,0,0,Yes,Self-employed,Rural,78.43,18.8,never smoked,0

Female,25,0,0,No,Private,Urban,82.77,36.3,Unknown,0

Male,49,0,0,Yes,Private,Rural,88.13,32.8,never smoked,0

Male,39,0,0,Yes,Private,Urban,102.77,35.8,smokes,0

Female,43,0,0,No,Private,Rural,101.75,26.7,smokes,0

Male,67,0,0,Yes,Private,Rural,113.34,26.3,formerly smoked,0

Male,48,0,0,Yes,Private,Urban,63.33,26.5,smokes,0

Female,38,0,0,Yes,Private,Urban,69.34,43.7,never smoked,0

Female,14,0,0,No,Private,Rural,118.81,24.7,Unknown,0

Male,63,0,0,Yes,Govt_job,Urban,66.13,46.2,never smoked,0

Male,20,0,0,No,Private,Urban,96.58,43.3,never smoked,0

Female,60,0,0,Yes,Private,Urban,57.89,30.9,formerly smoked,0

Female,13,0,0,No,children,Rural,78.38,38.7,Unknown,0

Male,34,0,0,No,Private,Urban,94.15,28.6,never smoked,0

Male,22,0,0,No,Private,Urban,89.53,30.2,Unknown,0

Female,46,0,0,Yes,Private,Urban,127.75,30.5,never smoked,0

Male,53,0,0,Yes,Self-employed,Rural,76.79,33.9,Unknown,0

Female,28,0,0,Yes,Private,Rural,141.15,28.6,never smoked,0

Female,62,0,0,Yes,Govt_job,Urban,88.63,24.5,never smoked,0

Female,11,0,0,No,children,Urban,130.15,17.2,Unknown,0

Male,66,0,0,No,Private,Urban,108.03,27.2,never smoked,0

Female,80,0,0,Yes,Self-employed,Rural,104.07,19.3,formerly smoked,0

Male,8,0,0,No,children,Urban,88.83,18.5,Unknown,0

Male,53,0,1,Yes,Private,Rural,197.79,32,Unknown,0

Male,63,1,1,Yes,Private,Urban,150.45,44.5,formerly smoked,0

Male,31,0,0,Yes,Private,Rural,100.39,37,never smoked,0

Male,43,0,0,Yes,Govt_job,Rural,79.92,30.8,formerly smoked,0

Male,79,0,1,Yes,Self-employed,Urban,96.79,24.7,Unknown,0

Male,8,0,0,No,children,Rural,86.84,18.3,Unknown,0

Male,38,0,0,Yes,Private,Rural,61.27,44,Unknown,0

Female,80,0,0,Yes,Self-employed,Rural,72.61,27.6,never smoked,0

Female,49,0,0,Yes,Private,Rural,82.41,45.4,smokes,0

Female,50,0,0,Yes,Govt_job,Urban,74.72,28.5,never smoked,0

Male,13,0,0,No,children,Rural,57.37,17.6,Unknown,0

Male,73,1,0,Yes,Private,Urban,88.34,27.5,never smoked,0

Male,60,0,0,Yes,Private,Rural,101.34,32.8,never smoked,0

Male,44,0,0,Yes,Private,Rural,119.01,29.5,never smoked,0

Female,79,0,0,No,Self-employed,Rural,71.98,36.4,never smoked,0

Female,5,0,0,No,children,Urban,84.91,26.1,Unknown,0

Female,51,0,0,Yes,Self-employed,Rural,91.63,35.3,Unknown,0

Male,5,0,0,No,children,Rural,145.71,18.1,Unknown,0

Female,65,0,0,Yes,Self-employed,Rural,167.31,27.1,never smoked,0

Female,81,0,0,Yes,Self-employed,Urban,63.65,23,Unknown,0

Male,44,0,0,Yes,Govt_job,Rural,87.49,26.6,never smoked,0

Female,25,0,0,No,Private,Rural,92.82,24.1,never smoked,0

Male,49,0,0,Yes,Govt_job,Urban,72.2,30.3,formerly smoked,0

Male,80,0,1,Yes,Govt_job,Rural,181.23,32.2,formerly smoked,0

Female,22,0,0,No,Private,Urban,63.37,26.5,never smoked,0

Female,61,0,0,Yes,Private,Urban,74.82,30.6,never smoked,0

Female,3,0,0,No,children,Rural,97.6,25.8,Unknown,0

Male,50,0,0,Yes,Private,Rural,93.04,41.9,smokes,0

Male,45,0,0,Yes,Private,Rural,97.12,29.2,never smoked,0

Male,55,1,0,Yes,Self-employed,Rural,82.81,44.3,never smoked,0

Female,21,0,0,No,Private,Urban,134.45,29.1,never smoked,0

Male,0.8,0,0,No,children,Rural,98.67,17.5,Unknown,0

Male,50,0,1,No,Private,Urban,85.82,31.9,never smoked,0

Male,73,0,1,Yes,Self-employed,Rural,189.45,32.2,never smoked,0

Female,58,0,1,Yes,Private,Rural,144.16,26,smokes,0

Male,30,0,0,No,Private,Rural,82.56,25.4,formerly smoked,0

Male,57,0,0,Yes,Self-employed,Urban,67.97,27.9,never smoked,0

Female,77,1,0,Yes,Self-employed,Urban,71.7,32.8,never smoked,0

Female,29,0,0,No,Private,Urban,72.02,34,formerly smoked,0

Female,46,0,0,Yes,Private,Urban,61.81,25.5,Unknown,0

Female,40,0,0,Yes,Private,Urban,103.09,35.6,Unknown,0

Male,29,0,0,No,Private,Urban,73.75,28.3,never smoked,0

Female,37,0,0,Yes,Private,Rural,156.7,36.9,never smoked,0

Female,35,0,0,Yes,Self-employed,Urban,70.87,22.1,formerly smoked,0

Male,54,0,0,Yes,Self-employed,Rural,138.47,31.5,never smoked,0

Female,56,0,1,Yes,Private,Rural,64.66,26.7,formerly smoked,0

Female,26,0,0,Yes,Private,Rural,91.88,24.9,formerly smoked,0

Female,75,0,0,Yes,Self-employed,Rural,74.79,32.4,never smoked,0

Female,44,0,0,No,Private,Rural,78.18,32.2,never smoked,0

Male,63,0,0,Yes,Private,Rural,77.82,30.3,Unknown,0

Female,17,0,0,No,Private,Rural,112.23,28.7,never smoked,0

Male,46,0,1,Yes,Private,Urban,178.76,24.1,never smoked,0

Female,47,0,0,Yes,Private,Rural,121.43,25.3,never smoked,0

Female,50,0,0,Yes,Private,Rural,94.22,24.8,never smoked,0

Male,76,1,0,Yes,Self-employed,Rural,197.58,34.8,formerly smoked,0

Male,47,0,0,Yes,Private,Rural,86.37,39.2,smokes,0

Female,47,0,0,Yes,Private,Urban,112.09,24.7,smokes,0

Male,49,0,0,Yes,Private,Urban,199.96,28.6,never smoked,0

Male,71,0,0,Yes,Private,Urban,85.33,27.7,never smoked,0

Female,65,0,0,No,Self-employed,Rural,86.33,33.1,never smoked,0

Female,40,0,0,No,Private,Urban,80.47,27.3,smokes,0

Female,82,1,0,Yes,Private,Urban,62.46,20.3,formerly smoked,0

Female,64,0,0,Yes,Govt_job,Rural,78.85,33.9,never smoked,0

Female,50,0,0,Yes,Self-employed,Rural,103.81,28.3,never smoked,0

Female,40,0,0,No,Private,Urban,78.04,32.4,smokes,0

Female,74,0,0,Yes,Self-employed,Rural,94.67,19.7,Unknown,0

Female,30,0,0,Yes,Govt_job,Rural,62.25,33.7,never smoked,0

Male,68,0,0,Yes,Private,Urban,155.17,35.5,never smoked,0

Male,6,0,0,No,children,Urban,83.28,20,Unknown,0

Male,4,0,0,No,children,Urban,132.41,16.3,Unknown,0

Female,60,0,0,Yes,Private,Rural,62.78,36.4,Unknown,0

Female,59,0,0,Yes,Private,Urban,89.96,28.1,Unknown,0

Female,72,0,0,Yes,Self-employed,Rural,103.25,26.9,formerly smoked,0

Female,62,0,0,Yes,Private,Rural,126.99,29.4,formerly smoked,0

Female,13,0,0,No,children,Rural,102.27,17.2,never smoked,0

Male,72,0,1,Yes,Self-employed,Rural,113.63,26.5,Unknown,0

Male,1.08,0,0,No,children,Urban,86.09,19.5,Unknown,0

Male,35,0,0,Yes,Private,Rural,108.08,30.6,formerly smoked,0

Male,54,0,0,Yes,Private,Urban,108.34,31.9,never smoked,0

Male,40,0,0,Yes,Private,Rural,56.07,26.6,never smoked,0

Female,3,0,0,No,children,Urban,82.91,19.9,Unknown,0

Male,2,0,0,No,children,Urban,94.75,18,Unknown,0

Male,41,0,0,Yes,Private,Rural,101.79,26.7,Unknown,0

Female,61,0,0,Yes,Private,Rural,88.41,25.3,formerly smoked,0

Male,58,0,0,Yes,Govt_job,Urban,101.05,31.4,Unknown,0

Male,17,0,0,No,Private,Urban,85.07,21.1,never smoked,0

Female,64,0,0,No,Govt_job,Rural,108.1,17.9,never smoked,0

Female,32,0,0,No,Self-employed,Rural,93.17,27.5,smokes,0

Female,42,0,0,Yes,Private,Urban,158.89,37.6,smokes,0

Male,39,0,0,Yes,Private,Rural,205.77,24.1,never smoked,0

Female,10,0,0,No,children,Urban,112.34,18.1,Unknown,0

Female,78,0,0,Yes,Private,Urban,119.03,31,never smoked,0

Female,68,0,0,Yes,Private,Urban,237.21,26.6,smokes,0

Female,24,0,0,Yes,Private,Urban,89.68,38.7,never smoked,0

Male,59,1,1,Yes,Private,Rural,246.53,27.2,formerly smoked,0

Female,45,0,0,Yes,Private,Urban,89.05,27.8,formerly smoked,0

Male,12,0,0,No,children,Rural,81.74,28.3,Unknown,0

Female,37,0,0,Yes,Private,Rural,98.12,27.5,never smoked,0

Female,43,0,0,Yes,Self-employed,Rural,105.59,43.3,smokes,0

Male,12,0,0,No,children,Rural,86,20.1,formerly smoked,0

Male,13,0,0,No,children,Urban,65.51,25.9,Unknown,0

Male,74,0,0,Yes,Self-employed,Urban,61.78,25.8,Unknown,0

Female,2,0,0,No,children,Rural,109.56,16.4,Unknown,0

Female,59,0,0,Yes,Self-employed,Urban,65.82,29.4,never smoked,0

Female,21,0,0,No,Private,Urban,90.16,28.9,smokes,0

Female,55,0,0,Yes,Private,Urban,64.45,26.7,never smoked,0

Female,14,0,0,No,children,Urban,91.85,27.8,never smoked,0

Female,53,0,0,Yes,Private,Rural,97.89,38.7,formerly smoked,0

Male,75,0,0,Yes,Govt_job,Rural,206.33,26.8,never smoked,0

Male,37,0,0,Yes,Govt_job,Rural,74.29,36.1,never smoked,0

Female,49,0,0,Yes,Private,,107.4,26.7,smokes,0

Male,55,0,0,Yes,Private,Urban,87.72,27,Unknown,0

Male,57,0,0,Yes,Private,Rural,85.99,21.2,Unknown,0

Female,45,0,0,Yes,Private,Urban,100.84,21,never smoked,0

Male,46,0,0,No,Private,Urban,124.61,37.4,Unknown,0

Female,12,0,0,No,children,Urban,86.55,26.5,Unknown,0

Male,12,0,0,No,children,Rural,90.42,28.9,Unknown,0

Female,77,0,0,Yes,Private,Rural,100.85,29.5,smokes,0

Male,6,0,0,No,children,Urban,87.74,17.7,Unknown,0

Male,76,1,0,Yes,Private,Rural,80.15,34.9,formerly smoked,0

Female,52,0,0,Yes,Private,Urban,107.29,28.1,never smoked,0

Male,62,0,0,Yes,Private,Urban,206.98,36.8,smokes,0

Male,82,1,0,Yes,Private,Rural,227.28,33.3,never smoked,0

Male,69,0,1,Yes,Private,Rural,80.43,29.2,Unknown,0

Male,78,0,1,Yes,Self-employed,Urban,228.7,34,Unknown,0

Male,37,0,0,Yes,Private,Urban,120.07,33.9,smokes,0

Female,50,0,0,Yes,Private,Rural,93.47,,never smoked,0

Female,41,0,0,Yes,Private,Urban,80.77,21.1,never smoked,0

Male,54,0,0,Yes,Private,Rural,150.27,38.2,smokes,0

Male,43,0,0,Yes,Private,Urban,81.94,27.7,smokes,0

Male,47,0,0,Yes,Private,Rural,93.55,31.4,never smoked,0

Male,44,0,0,Yes,Private,Urban,99.34,33.1,never smoked,0

Female,37,0,0,Yes,Private,Urban,70.75,35.8,Unknown,0

Male,9,0,0,No,children,Urban,83.83,27.1,Unknown,0

Female,11,0,0,No,children,Urban,73.66,20.5,never smoked,0

Male,56,1,0,Yes,Private,Urban,102.37,35.6,never smoked,0

Female,5,0,0,No,children,Rural,77.83,15.6,Unknown,0

Female,17,0,0,No,Private,Urban,70.01,43,Unknown,0

Female,6,0,0,No,children,Rural,119.88,17.8,Unknown,0

Female,57,0,0,Yes,Private,Rural,169.97,25.8,never smoked,0

Male,0,0,No,Private,Rural,123.79,20.5,Unknown,0

Male,2,0,0,No,children,Urban,80.98,19.9,Unknown,0

,76,0,0,Yes,Govt_job,Rural,84.21,24.4,never smoked,0

Male,1,0,0,No,children,Urban,101.31,18.3,Unknown,0

Male,54,0,0,Yes,Private,Urban,86.26,35.1,formerly smoked,0

Female,45,0,0,Yes,Private,Urban,92.86,35.1,formerly smoked,0

Female,54,0,1,Yes,Private,Urban,140.28,37.1,formerly smoked,0

Male,67,0,0,Yes,Govt_job,Rural,244.28,29.4,formerly smoked,0

Male,53,0,0,Yes,Private,Urban,124.16,31.7,never smoked,0

Male,47,0,0,Yes,Private,Rural,93.55,31.4,never smoked,0

Male,44,0,0,Yes,Private,Urban,99.34,33.1,never smoked